第3回　ニューラルネットワーク基礎

講義講師：小林由弥
演習講師：池田悠也

許諾なく撮影や第三者
への開示を禁止します

©Yutaka Matsuo. All rights reserved.

全体の位置づけ

#	タイトル
1	人工知能の概要とディープラーニングの意義
2	機械学習基礎
3	ニューラルネットワーク基礎
4	ニューラルネットワークの最適化・正則化
5	畳み込みニューラルネットワーク
6	深層学習と画像認識
7	回帰結合型ニューラルネットワークと系列データ処理
8	Transformer基礎
9	大規模言語モデルと自然言語処理
10	表現学習と自己教師あり学習
11	深層強化学習
12	深層生成モデル

©Yutaka Matsuo. All rights reserved.

講師紹介

担当講師: 小林 由弥

~修士:
神経の数理モデル、脳波解析など @神保・小谷研究室

博士課程:
深層生成モデル・compositionality(構成性)の研究など @松尾研

~現在:
卒業後はソニーコンピュータサイエンス研究所 (Sony CSL)に
→ 現在はSonyAI (research scientist)に所属

松尾研の講義や、基礎研究プロジェクトへの一時参加など

©Yutaka Matsuo. All rights reserved.

今回の目的と目標

- **目的**
  - 基本的なニューラルネットワークである，多層パーセプトロン（MLP）を理解する
    - 基本的な構成要素と学習（最適化）方法を理解する
  - ディープニューラルネットワーク（深層学習, DNN）へ拡張する方法を理解する
    - 要素技術や学習方法を理解する（活性化関数・確率的勾配降下法など）
- **目標**
  - MLPは入力の非線形分離が可能であることを説明できる
    - ロジスティック回帰による入力の境界分離は線形
  - MLPに関する基本的な概念を説明できる
    - 重み，ユニット，隠れ層，出力層，誤差関数，活性化関数など
  - 損失関数に対応する，重みの勾配を求めることができる（誤差逆伝播法・連鎖律）
  - MLPを多層化した時に起こる問題点と，その対策について説明できる

©Yutaka Matsuo. All rights reserved.

目次

- 前回までのおさらいと非線形問題
- MLP（多層パーセプトロン）の基本的な考え方と構成要素
- MLPの設定項目（ハイパーパラメータ・アーキテクチャ・誤差関数）
- 誤差逆伝播法と微分の連鎖律
- 多層化する際の問題点

©Yutaka Matsuo. All rights reserved.

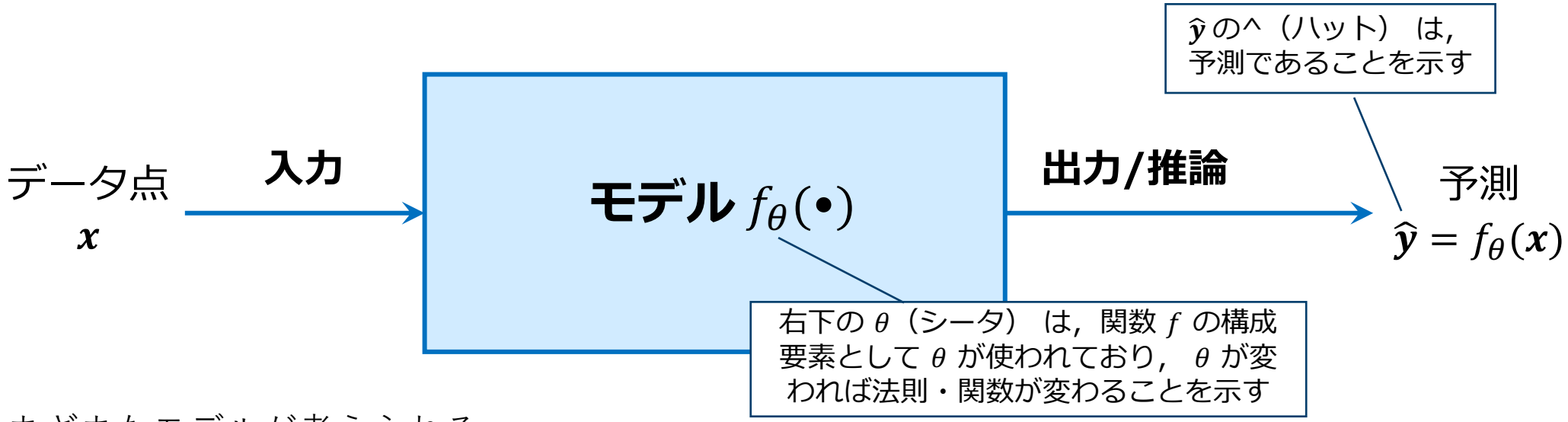
目次

- **前回までのおさらいと非線形問題**
- MLP（多層パーセプトロン）の基本的な考え方と構成要素
- MLPの設定項目（ハイパーパラメータ・アーキテクチャ・誤差関数）
- 誤差逆伝播法と微分の連鎖律
- 多層化する際の問題点

©Yutaka Matsuo. All rights reserved.

モデルとは何か？ （前回の振り返り）

- モデルとは、入力を元に出力を予測するための法則・関数の記述である

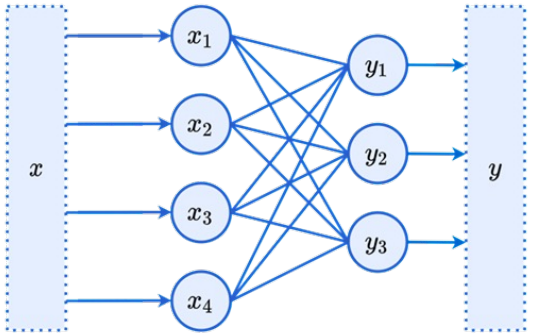


さまざまなモデルが考えられる

単回帰分析の例：$f_\theta(\boldsymbol{x}) = \alpha + \beta x$ （この場合 $\theta = (\alpha, \beta)$）

単層パーセプトロンの例（この場合 $\theta = (\boldsymbol{W}, \boldsymbol{b})$）

$$f_\theta(\boldsymbol{x}) = \boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b}$$

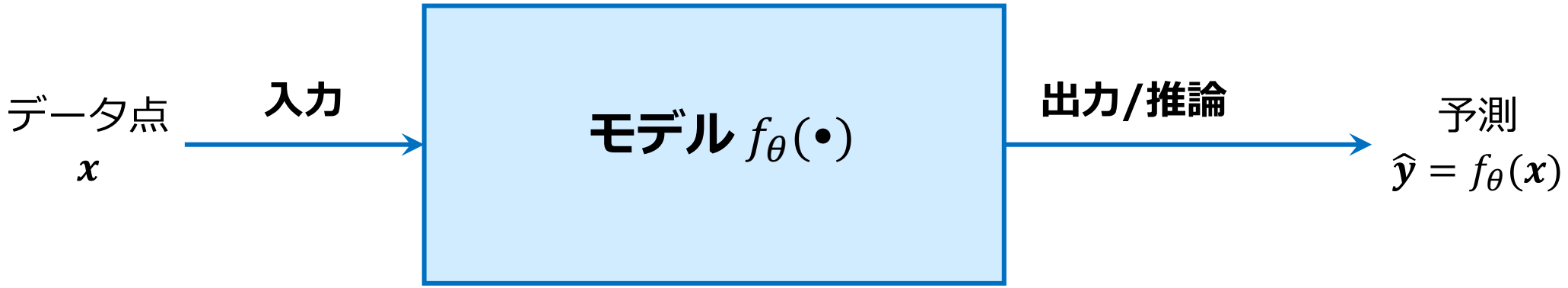


©Yutaka Matsuo. All rights reserved.

機械学習モデルの推論と学習（前回の振り返り）

- 推論：入力 $\boldsymbol{x}$ に対して対応する出力の予測 $\hat{\boldsymbol{y}} = f_\theta(\boldsymbol{x})$ を返す

- 学習：出力の予測 $\hat{\boldsymbol{y}}$ が期待する結果に近づくようにモデルを調整する
  - モデル f_θ の予測 $\hat{\boldsymbol{y}}$ と正解は必ずしも一致しない
  - 学習済みのモデル f_θ の予測 $\hat{\boldsymbol{y}}$ は正解に近くなることが期待される

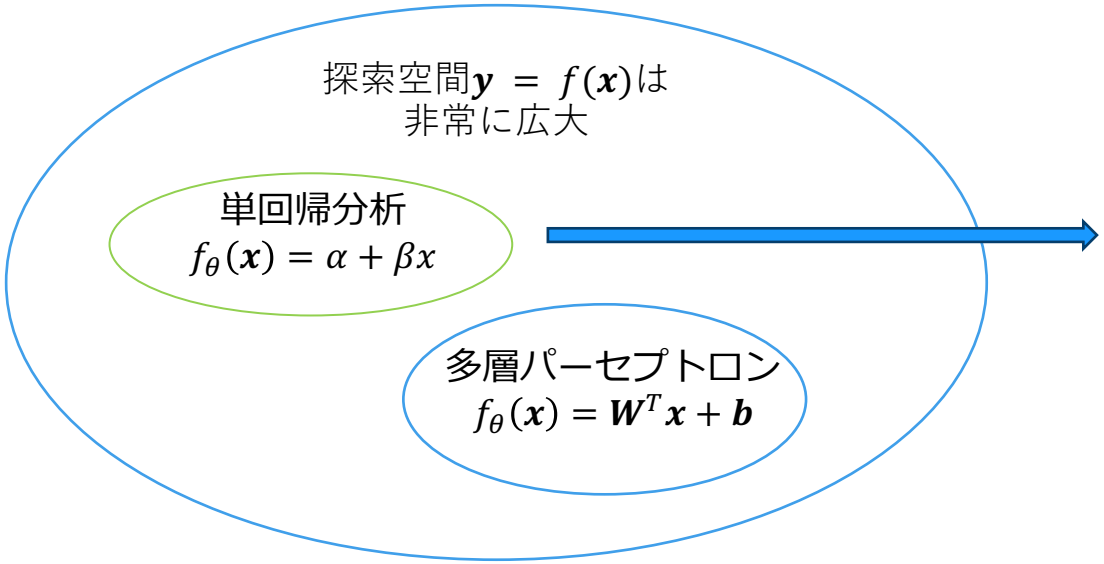


©Yutaka Matsuo. All rights reserved.

機械学習はモデル探索（前回の振り返り）

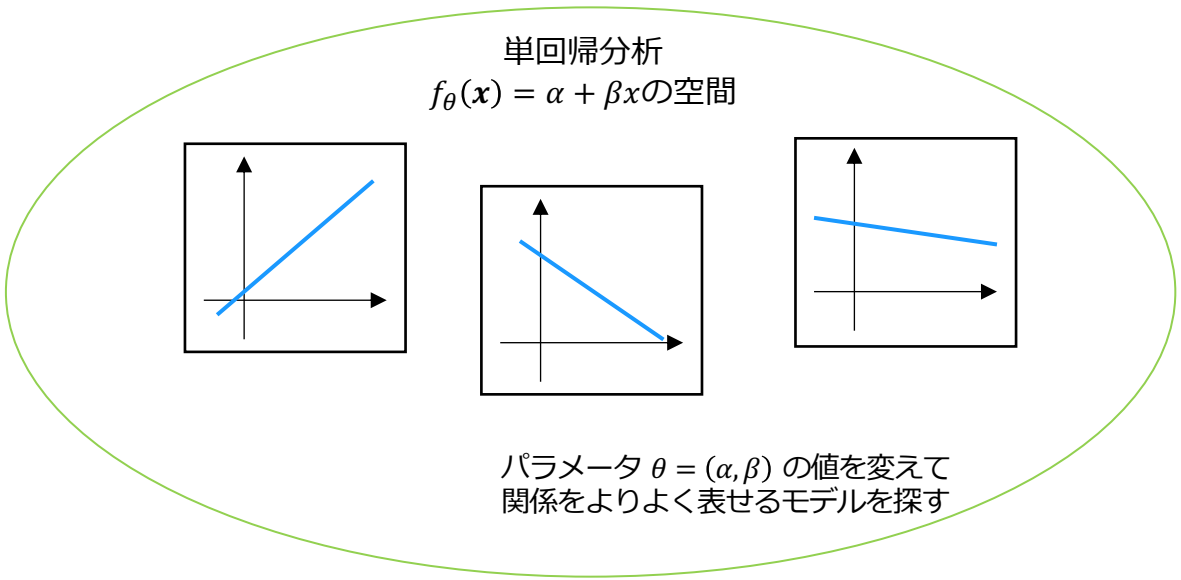
- 学習とは入出力$(\boldsymbol{x}, \boldsymbol{y})$ の関係 $\boldsymbol{y} = f(\boldsymbol{x})$ をよりよく表せる関数 f の探索である
- 探索空間を有限にするために，モデルをパラメトリック(*)に記述して**探索対象とする関数の空間(範囲)**を限定し，パラメータを調整することで探索を行う

① 探索空間を限定
まずハイパーパラメータを設定することによって
モデル f_θを選択し、探索空間を狭める



関係 $\boldsymbol{y} = f(\boldsymbol{x})$ をよく表せる関数が見つかりそうな
モデル f_θ を選択することが重要

② パラメータを調整
モデル f_θを選択した後は，最も妥当な
パラメータ $\boldsymbol{\theta}$ をデータに基づいて探す（学習）



(*) パラメータによって記述されるモデルをパラメトリックモデル（Parametric Model）と呼ぶ
パラメータを用いて記述されないモデルはノンパラメトリックモデル（Nonparametric Model）と呼ぶ

©Yutaka Matsuo. All rights reserved.

機械学習の具体例（前回の振り返り）

- 入出力関係を規定する関数 f を「学習」する

画像認識



x

⇨ 関数 f ⇨

Tiger: 1
Cat: 0
Dog: 0

y

画像キャプション



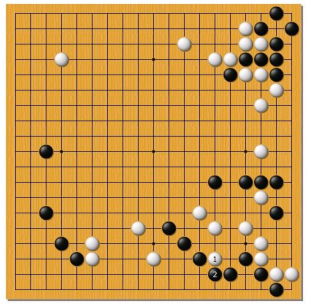
x

⇨ 関数 f ⇨

A man in a black shirt is playing the guitar.

y

囲碁AI



x

⇨ 関数 f ⇨

Win: 1
Lose: 0

y

機械翻訳

It is my pleasure to see you.

x

⇨ 関数 f ⇨

お会い出来て光栄です

y

©Yutaka Matsuo. All rights reserved.

そもそも「学習」って？

- 学習とは [Mitchell 1997]
  “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ”

「コンピュータプログラムは，性能指標 P で測定されるタスク T における性能が経験 E により改善される場合，そのタスク T のクラスおよび性能指標 P に関して経験 E から学習すると言われている.（訳は「深層学習」より）」

- 性能指標 P
- 経験 E
- タスク T

©Yutaka Matsuo. All rights reserved.

そもそも「学習」って？

- 学習とは [Mitchell 1997]
  “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E. ”

「コンピュータプログラムは，性能指標 P で測定されるタスク T における性能が経験 E により改善される場合，そのタスク T のクラスおよび性能指標 P に関して経験 E から学習すると言われている.（訳は「深層学習」より）」

- 性能指標 P：システムの性能を測る指標 例：認識精度，対数尤度
- 経験 E：学習に使えるデータ集合 例：画像とラベルのペアの集合（教師あり学習）
- タスク T：達成したい問題設定 例：画像分類，音声分類，etc...

（コンピュータプログラム = 機械学習モデル）

©Yutaka Matsuo. All rights reserved.

学習モデル

- 次に「タスク T」や「経験 E」，「性能 P」，
  「学習するプログラム（モデル）」の具体例を見ていく

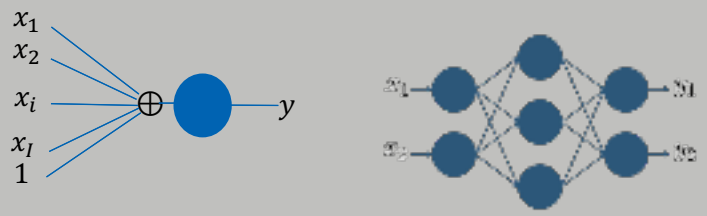
機械学習：データ集合からパターンや表現，判断ルールなどを自動で獲得する手法

- 線形回帰・ロジスティック回帰
- サポートベクターマシン（SVM）
- ランダムフォレスト
- k-means法など

ニューラルネットワーク（Neural Network ; NN）

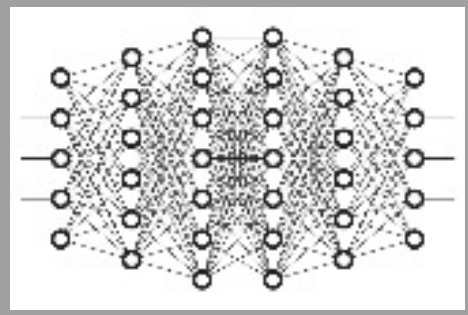
：脳の神経回路を模した学習モデル

単純パーセプトロン　多層パーセプトロン



深層学習（Deep Learning ; DL）

多層化したニューラルネットワーク



©Yutaka Matsuo. All rights reserved.

学習モデル

- 次に「タスク T」や「経験 E」，「性能 P」，
  「学習するプログラム（モデル）」の具体例を見ていく

機械学習：データ集合からパターンや表現，判断ルールなどを自動で獲得する手法

前回の内容

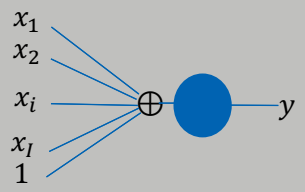
- 線形回帰・ロジスティック回帰
- サポートベクターマシン（SVM）
- ランダムフォレスト
- k-means法など

ニューラルネットワーク（Neural Network ; NN）

：脳の神経回路を模した学習モデル

今回の内容

単純パーセプトロン　多層パーセプトロン

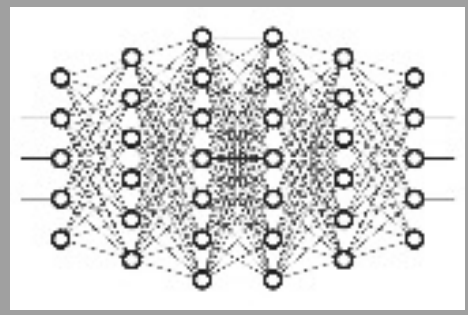




深層学習（Deep Learning ; DL）

多層化したニューラルネットワーク

主に次回以降

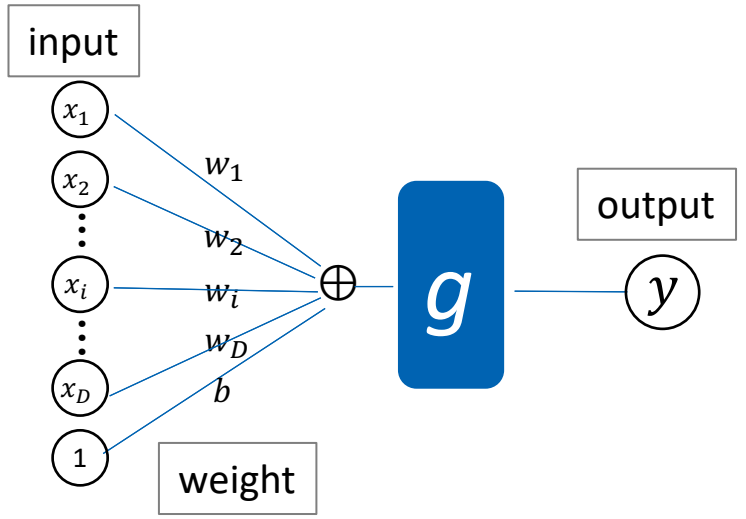


©Yutaka Matsuo. All rights reserved.

学習モデルの例：単純パーセプトロン

- **パーセプトロン（Perceptron）** [Rosenblatt 1957]
  - **コネクショニズム（Connectionism）：**（ニューラルネットワークによって知能を実現しようとする）の元祖
  - 初めての学習ができるニューラルネットワーク
    - 人工ニューロンの元祖はMcCulloch & Pittsによる形式ニューロン [McCulloch & Pitts 1943]

$$y = g(\sum_{i=1}^{D} w_i x_i + b) = g(\boldsymbol{w}^T \boldsymbol{x} + b)$$



  - $\boldsymbol{x} = (x_1, \ldots, x_D)^T$ は入力，y は出力
  - $\boldsymbol{w} = (w_1, \ldots, w_D)^T$ と b がモデルのパラメータ $\boldsymbol{\theta}$
  - g は任意の非線形関数（活性化関数）
    - 単純パーセプトロンではステップ関数：0未満に対し0を，0以上に対して1を返す関数
  - ニューラルネットワークの基本構造である
  - ※元々は2層（S->A->C）で構成されているが，1層目はランダム結合なので，ここでは単層モデルとして説明

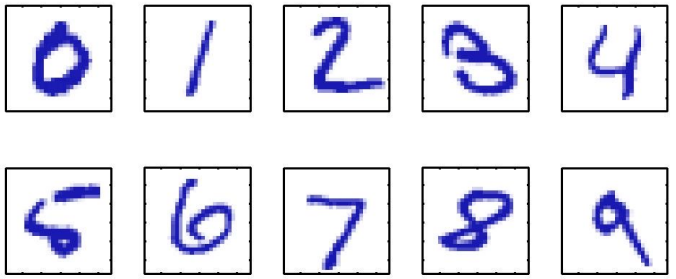
©Yutaka Matsuo. All rights reserved.

具体例①：手書き数字画像の分類問題

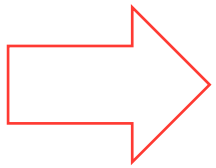
- タスクT：
  - 数字画像 $\boldsymbol{x}$ とラベル y（0~9）の関係を獲得する
  - 予測したい目標 y が離散値の場合，分類問題と呼ばれる

- 経験E：
  - $\boldsymbol{x}$ と y のペアの集合（教師ありデータ集合）

- 性能P：
  - 未知の数字画像 $\boldsymbol{x}$ に対して予測したラベル $\hat{y}$ の正解率（目的関数，誤差関数）



数字画像（行列 $\boldsymbol{x}$）



0 1 2 3 4
5 6 7 8 9

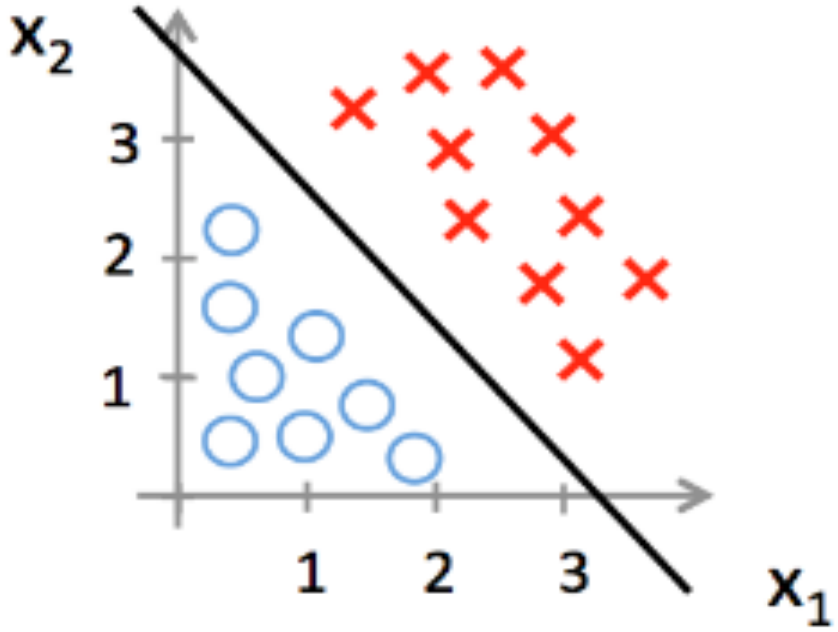
ラベル（整数 y）

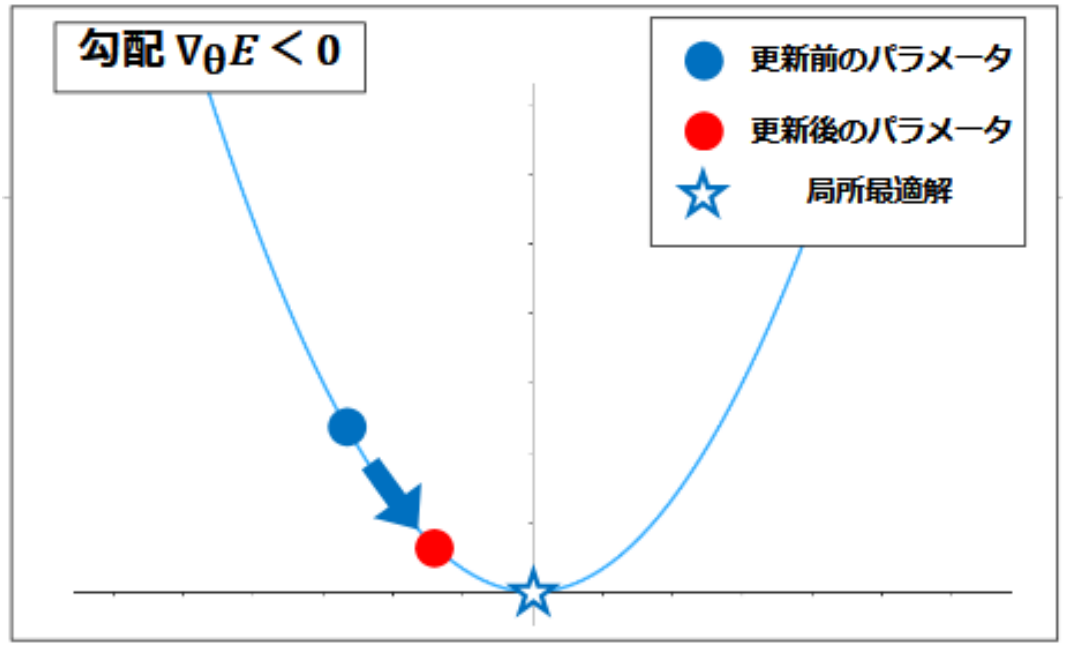
©Yutaka Matsuo. All rights reserved.

具体例①：ロジスティック回帰による解法

- モデルとして $\boldsymbol{y} = \mathrm{softmax}(\boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b})$ を仮定
  - 単純パーセプトロンの活性化関数をソフトマックスにして出力を[0, 1]（確率）にしたものと言える

- クロスエントロピー誤差関数をパラメータ（$\boldsymbol{W}$と$\boldsymbol{b}$）について最小化
  - 勾配降下法（反復解法）で最適なパラメータを求める（右図）





©Yutaka Matsuo. All rights reserved.

具体例②：住宅価格予測（回帰問題）

- タスクT：
  - エリア，大きさ，築年数など $\boldsymbol{x}$ から住宅価格 y を予測する
  - 予測したい目標 y が連続値の場合，回帰問題と呼ばれる
- 経験E：
  - $\boldsymbol{x}$ と y のペアの集合（教師ありデータ集合）
- 性能P：
  - 未知の住宅情報 $\boldsymbol{x}$ に対する価格予測 $\hat{y}$ の精度

train.csv (449.88 KB) 20 of 81 columns

LotArea	Street	Alley	LotShape	LandContour	Utilities
1.3k – 2k	Pave 100%	NA 94%	Reg 63%	Lvl 90%	AllPub
	Grvl 0%	Grvl 3%	IR1 33%	Bnk 4%	NoSeWa
	Other (1) 0%	Other (2) 3%	Other (3) 4%	Other (3) 6%	Other (1)
8450	Pave	NA	Reg	Lvl	AllPub
9600	Pave	NA	Reg	Lvl	AllPub
11250	Pave	NA	IR1	Lvl	AllPub
9550	Pave	NA	IR1	Lvl	AllPub
14260	Pave	NA	IR1	Lvl	AllPub
14115	Pave	NA	IR1	Lvl	AllPub
10084	Pave	NA	Reg	Lvl	AllPub
10382	Pave	NA	IR1	Lvl	AllPub
6120	Pave	NA	Reg	Lvl	AllPub
7420	Pave	NA	Reg	Lvl	AllPub
11200	Pave	NA	Reg	Lvl	AllPub
11924	Pave	NA	IR1	Lvl	AllPub
12968	Pave	NA	IR2	Lvl	AllPub
10652	Pave	NA	IR1	Lvl	AllPub
10920	Pave	NA	IR1	Lvl	AllPub
6120	Pave	NA	Reg	Lvl	AllPub
11241	Pave	NA	IR1	Lvl	AllPub
10791	Pave	NA	Reg	Lvl	AllPub
13695	Pave	NA	Reg	Lvl	AllPub

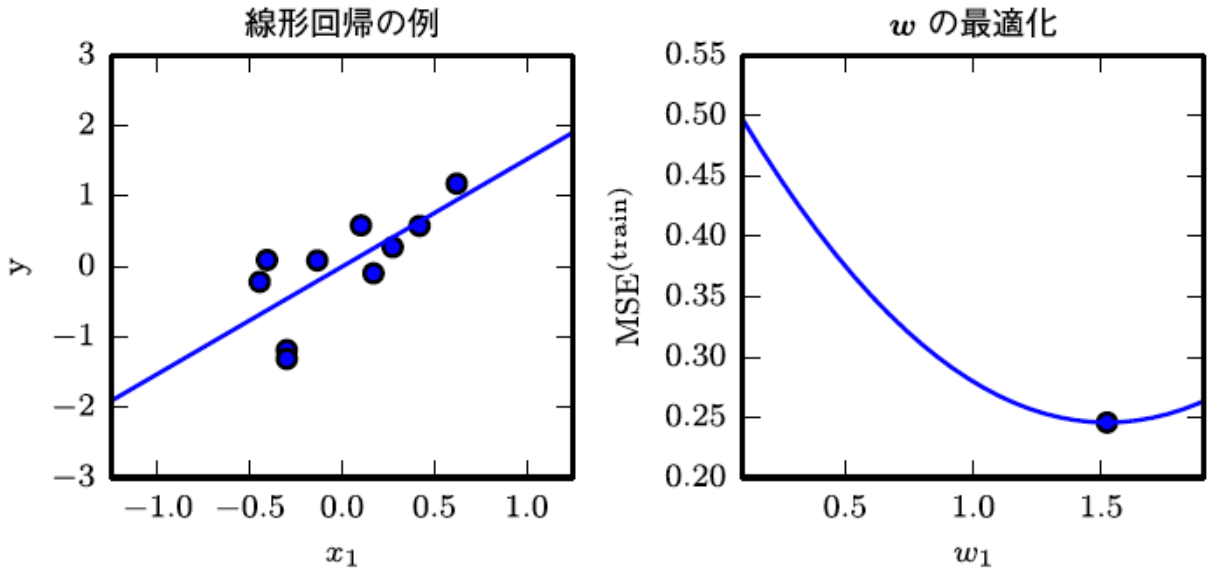
Kaggle:House Prices - Advanced Regression Techniques

©Yutaka Matsuo. All rights reserved.

具体例②：線形回帰による解法

- モデルとして $y = \boldsymbol{w}^T \boldsymbol{x} + b$ を仮定
  - $\boldsymbol{w}$ と b がモデルのパラメータ
  - 活性化関数 g が恒等写像の単純パーセプトロンとも言える

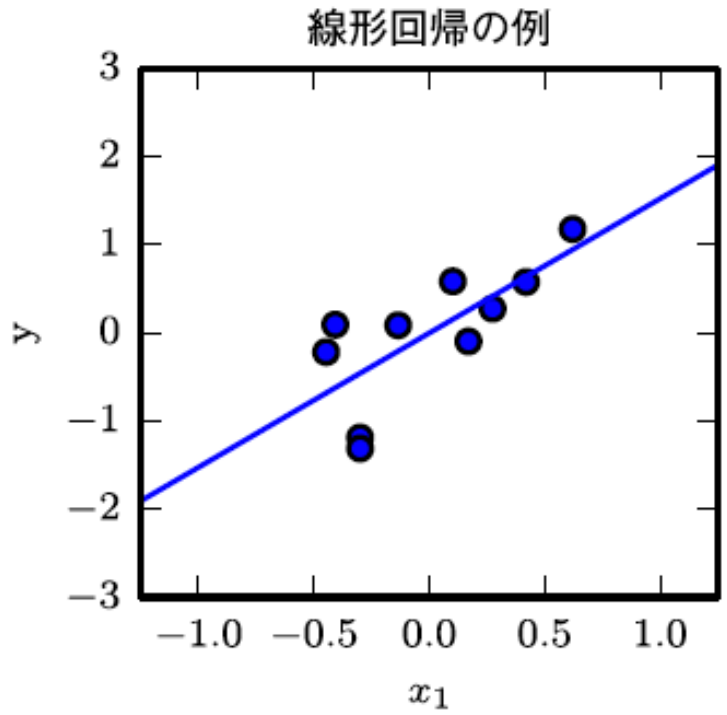
- 平均二乗誤差（MSE）関数をパラメータ（$\boldsymbol{w}$ と b）について最小化
  - 正規方程式を解いて最適なパラメータを求める（閉形式の解がある）
    - 正規方程式・閉形式については後述



©Yutaka Matsuo. All rights reserved.

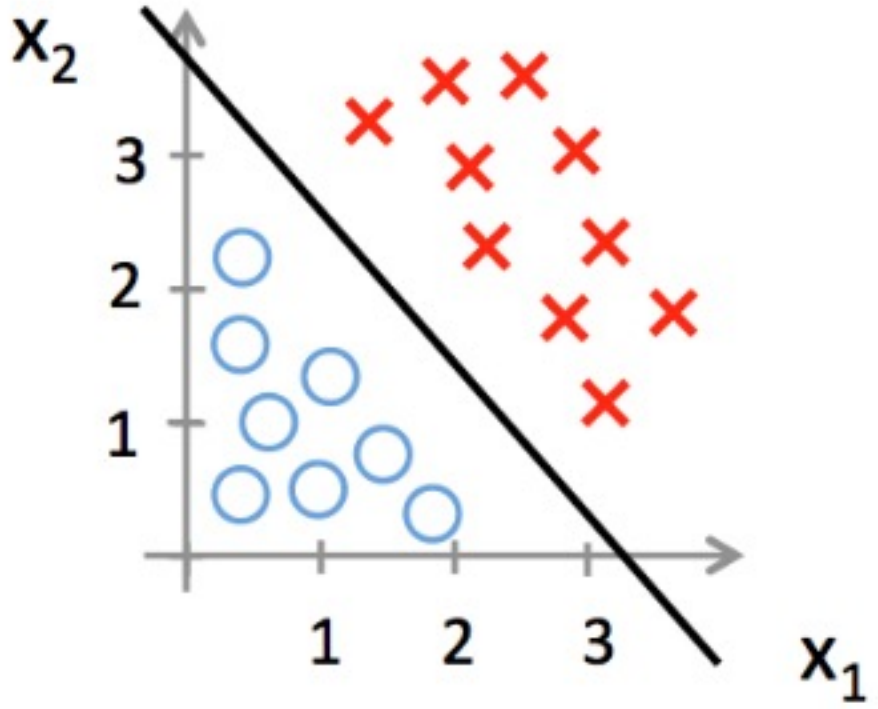
線形回帰&ロジスティック回帰の課題：「線形」の仮定

- 1本の直線で0と1を分離できる ＝ **線形分離可能（Linearly Separable）**



$$y = \boldsymbol{w}^T \boldsymbol{x} + b$$

真っ直ぐな線（超平面）



$$y = \sigma(\boldsymbol{w}^T \boldsymbol{x} + b)$$

真っ直ぐな線（超平面）

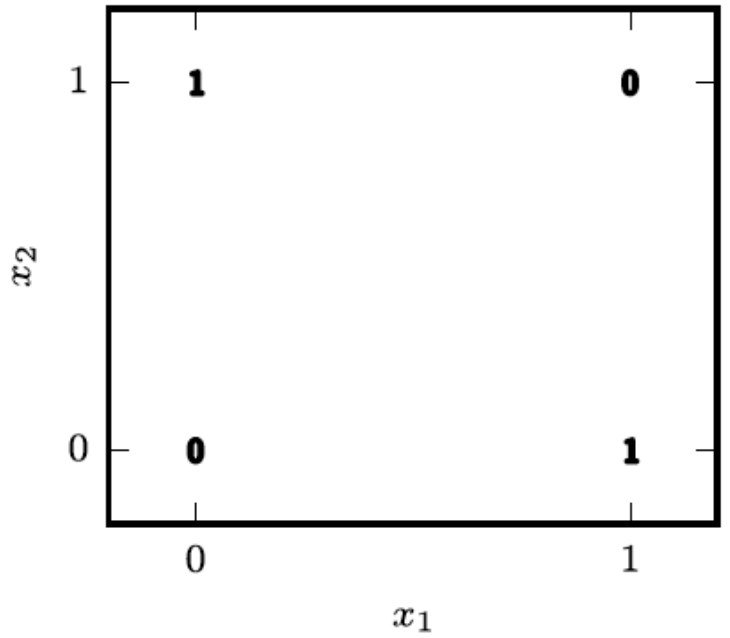
©Yutaka Matsuo. All rights reserved.

線形分離の限界：XOR問題

- XOR：排他的論理和
  - 2つの入力（x_1, x_2）の値が
    同じときは0，それ以外は1を割り当てる演算

    (0, 0) → 0　(0, 1) → 1　(1, 0) → 1　(1, 1) → 0

[XORの入出力]



- **線形分離不可能（Linearly Inseparable）**
  - どんな直線でも0と1を分離不可

- Minskyらによる指摘 [Minsky+ 1969]
  - 信用割当問題（Credit Assignment Problem）
    - 「各パラメータが予測にどのように役立ち，どれをどう修正すれば精度が上がるか分からない」という問題
    - 多層化が困難に
  - 第1次ニューラルネットワークブームの終焉につながる

©Yutaka Matsuo. All rights reserved.

線形分離可能？？

- 非線形だし，連続ですらないと考えられる（特に言語の場合など）

画像認識



x

関数 f

Tiger: 1
Cat: 0
Dog: 0

y

画像キャプション



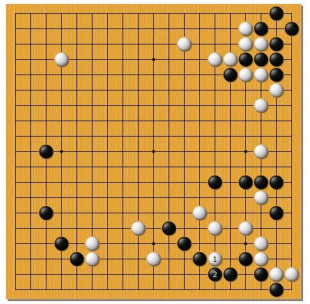
x

関数 f

A man in a black shirt is playing the guitar.

y

囲碁AI



x

関数 f

Win: 1
Lose: 0

y

機械翻訳

It is my pleasure to see you.

x

関数 f

お会い出来て光栄です

y

©Yutaka Matsuo. All rights reserved.

補足：ロジスティック回帰は線形分離しかできない？

Q: 非線形関数を通しているのに線形分離しかできないのか？
（σ はシグモイド関数）

$$y = \sigma(\boldsymbol{w}^T\boldsymbol{x} + b)$$

分離の閾値を0.5（任意）とすると，$y = 0.5$が分離平面（直線）なので

$$0.5 = \frac{1}{1+\exp[-(\boldsymbol{w}^T\boldsymbol{x}+b)]} \quad \rightarrow \quad \exp[-(\boldsymbol{w}^T\boldsymbol{x} + b)] = 1$$

よって $\boldsymbol{w}^T\boldsymbol{x} + b = 0$ → 直線の式になっている

©Yutaka Matsuo. All rights reserved.

ここまでのまとめ

- おさらい
  - 「学習」って何？
  - 機械学習の登場人物（問題設定）
    - 学習モデル，タスク T，評価指標 P，経験 E
  - 線形モデル
    - 線形回帰
    - ロジスティック回帰

- 線形モデルの課題：非線形な関係を学習できない
  - XOR問題
  - 現実のデータは非線形（連続ですらない）

©Yutaka Matsuo. All rights reserved.

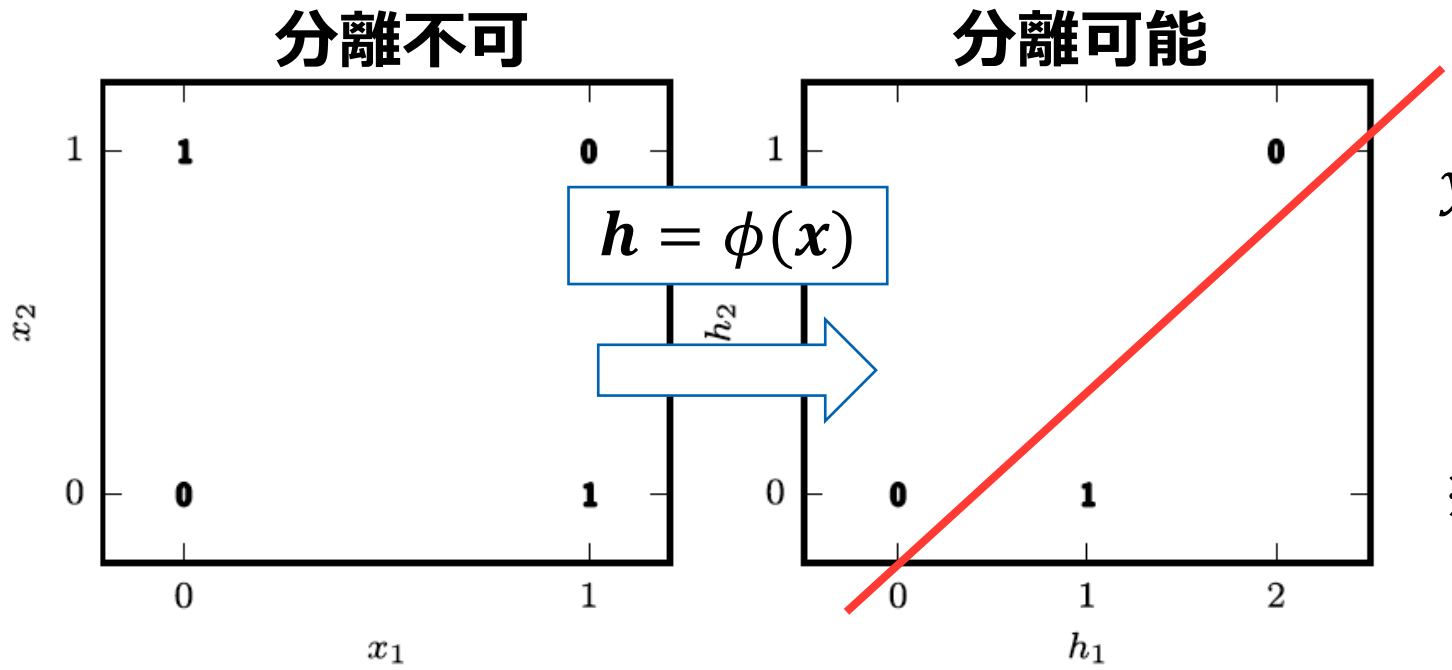
目次

- 前回までのおさらいと非線形問題
- **MLP（多層パーセプトロン）の基本的な考え方と構成要素**
- MLPの設定項目（ハイパーパラメータ・アーキテクチャ・誤差関数）
- 誤差逆伝播法と微分の連鎖律
- 多層化する際の問題点

©Yutaka Matsuo. All rights reserved.

非線形問題の解き方

- 入力空間 $\boldsymbol{x}$ を関数 ϕ によって線形分離可能な空間 $\boldsymbol{h}$ に写像する



$$y = f(\boldsymbol{h}; \boldsymbol{w}, b)$$
$$= g(\boldsymbol{w}^T \boldsymbol{h} + b)$$

→学習

※ g は活性化関数

写像 ϕ をどう設計するか？

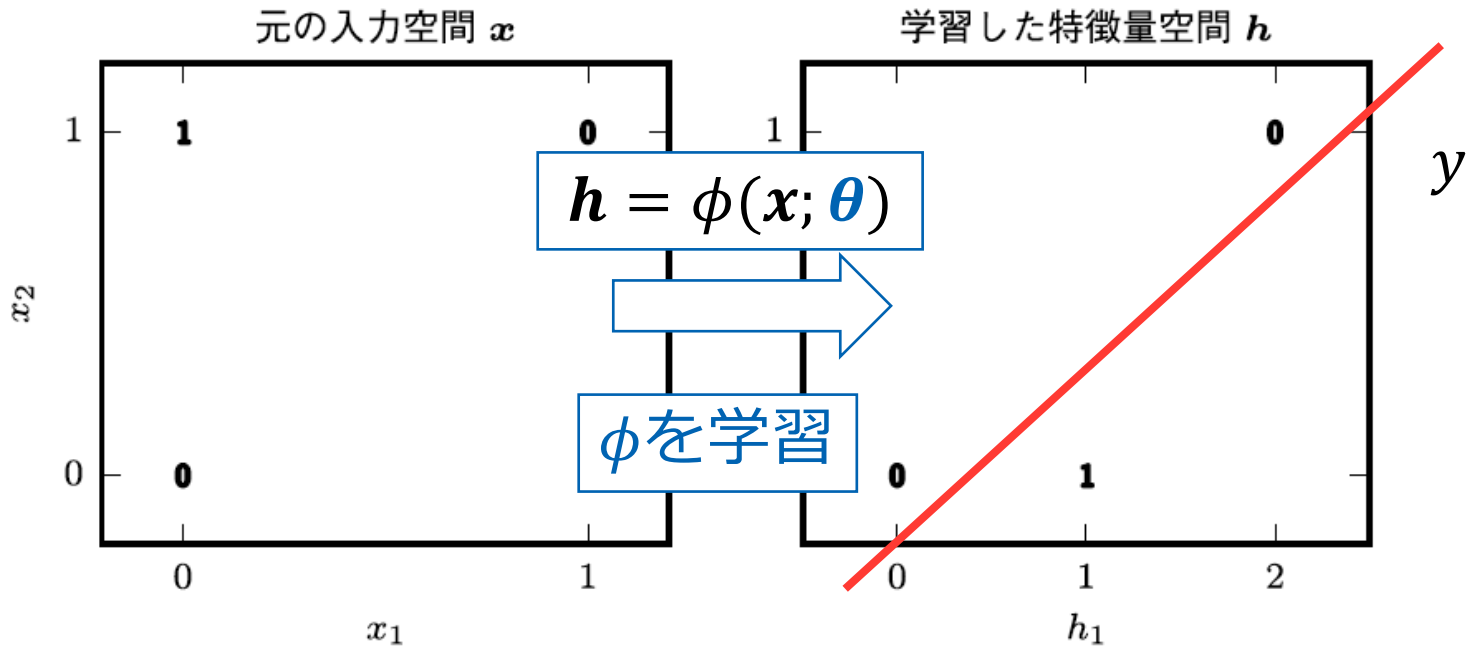
※ 写像 ≒ 関数

©Yutaka Matsuo. All rights reserved.

方法：写像関数自体をデータから学習

- 写像関数をパラメータ $\boldsymbol{\theta}$ を持つと考え，そのパラメータをデータから学習
  - つまり，識別関数 f と写像する関数 ϕ の両方を学習する

- 写像関数 ϕ・識別関数 f を共にパーセプトロンとしたモデル
  →**多層パーセプトロン（Multi-Layer Perceptron; MLP）**



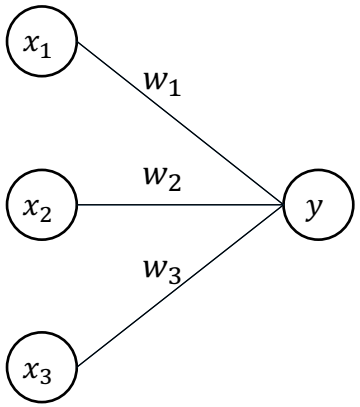
$$y = f(\boldsymbol{h}; \boldsymbol{w}, b)$$
$$= g(\boldsymbol{w}^T \boldsymbol{h} + b)$$

f を学習

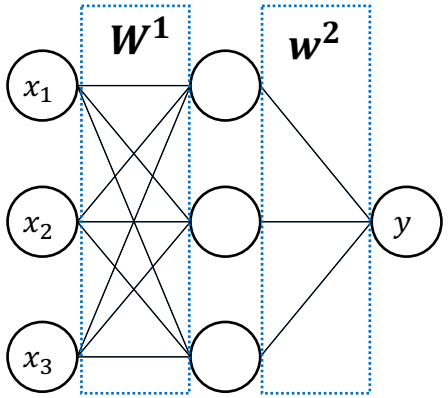
©Yutaka Matsuo. All rights reserved.

多層パーセプトロン

- 多層パーセプトロン
  - パーセプトロンを多層にすることで非線形な関数をモデル化
  - 順伝播型ニューラルネットワークとも呼ばれる
    - 入力から出力への「順伝播の流れ」を持つニューラルネットワーク



2層化



(注) 式では活性化関数とバイアスを明示しているが、図では省略している。
簡素化のため、図は以後この書き方とする。

$y = g(\boldsymbol{w}^T\boldsymbol{x} + b)$ ※$\boldsymbol{w} = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}$

単純パーセプトロン

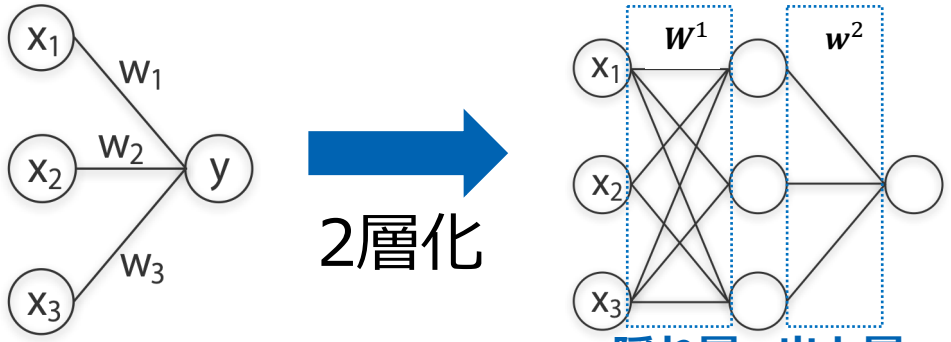
$y = g_2\left((\boldsymbol{w}^2)^T g_1\left((\boldsymbol{W}^1)^T\boldsymbol{x} + \boldsymbol{b}_1\right) + b_2\right)$

2層構造になっている（MLP）

©Yutaka Matsuo. All rights reserved.

MLPの構成要素：隠れ層

- 入力と出力の間に関数（パーセプトロン）が追加されている
  - 変数を変換する過程（関数）を**層（Layer）**と呼ぶ
    - ○が並んだ列の間の部分．左から第1層，第2層，…と数える．最後の層を**出力層（Output Layer）**という
  - 層の数を，モデルの**深さ（Depth）**と呼ぶ
  - 図中の○を**ユニット（Unit）**と呼ぶ（こちらを層と呼ぶ文献もある）
- 追加された層を，**隠れ層（Hidden Layer）**あるいは**中間層（Intermediate Layer）**と呼ぶ
  - 隠れ層の存在によって，出力への変換が複数回となる



perceptron

$$y = g(\boldsymbol{w}^T\boldsymbol{x} + b)$$

MLP

$$y = g_2\left((\boldsymbol{w}^2)^T g_1\left((\boldsymbol{W}^1)^T\boldsymbol{x} + \boldsymbol{b}_1\right) + b_2\right)$$

©Yutaka Matsuo. All rights reserved.

MLPの構成要素：活性化関数

- 主な活性化関数の式とグラフ

- シグモイド関数　0～1の範囲で単調増加

$$\mathrm{sigmoid(h)} = \frac{1}{1+\exp(-h)}$$

- **双曲線正接関数（tanh）**　-1～1の範囲で単調増加

$$\tanh(\mathrm{h}) = \frac{e^{h}-e^{-h}}{e^{h}+e^{-h}} = 2\mathrm{sigmoid}(2h) - 1$$

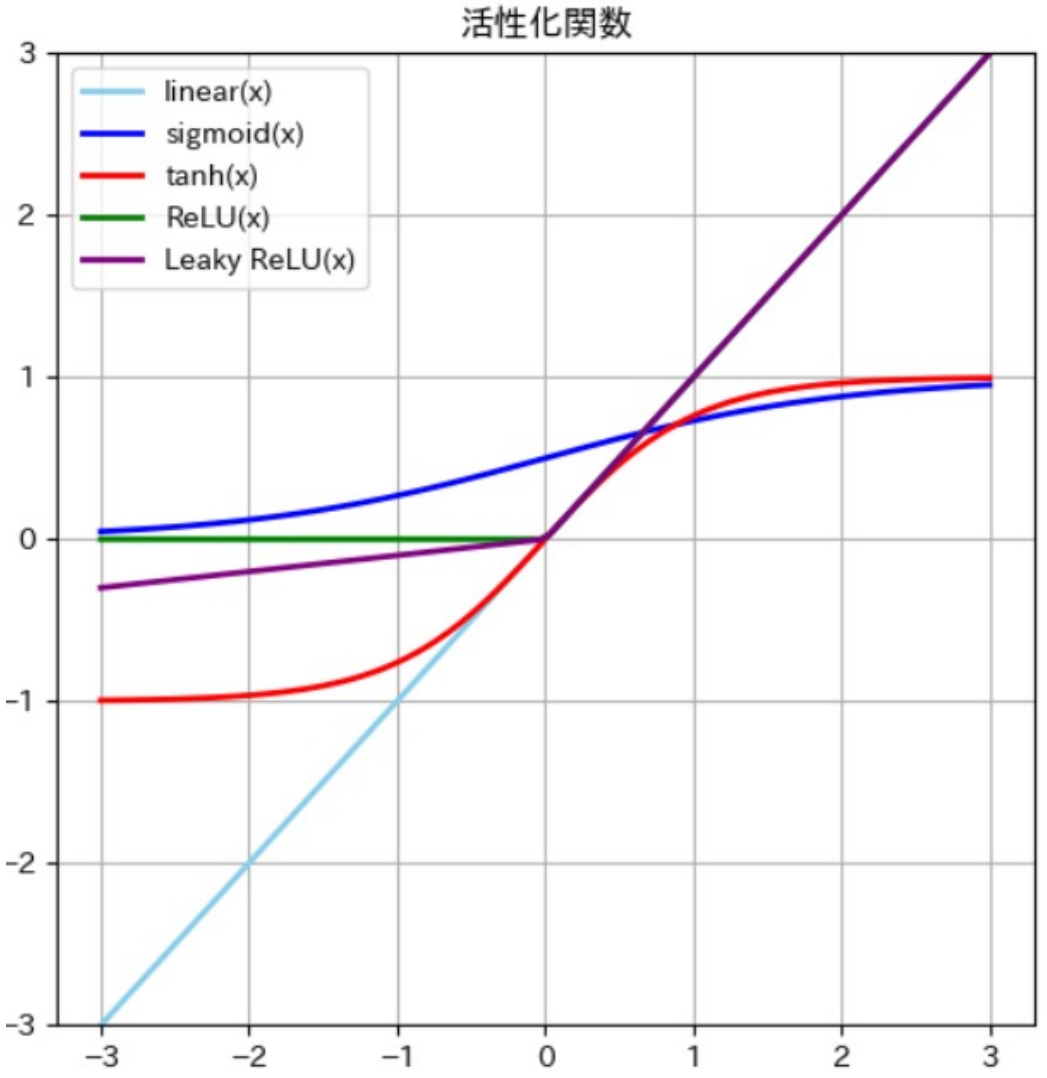
- **正規化線形関数（Rectified Linear Unit；ReLU）** [Xavier+ 2011]　負の入力は0に、正の入力はそのまま返す関数

$$\mathrm{ReLU(h)} = \max(\mathrm{h}, 0)$$

- **Leaky ReLU** [Mass+ 2013]　負の入力はα倍に縮小して、正の入力はそのまま返す関数

$$\text{Leaky ReLU}(h) = \max(\alpha h, h)$$

通常$\alpha = 0.01$とされる
（図では可視性のため0.1としている）

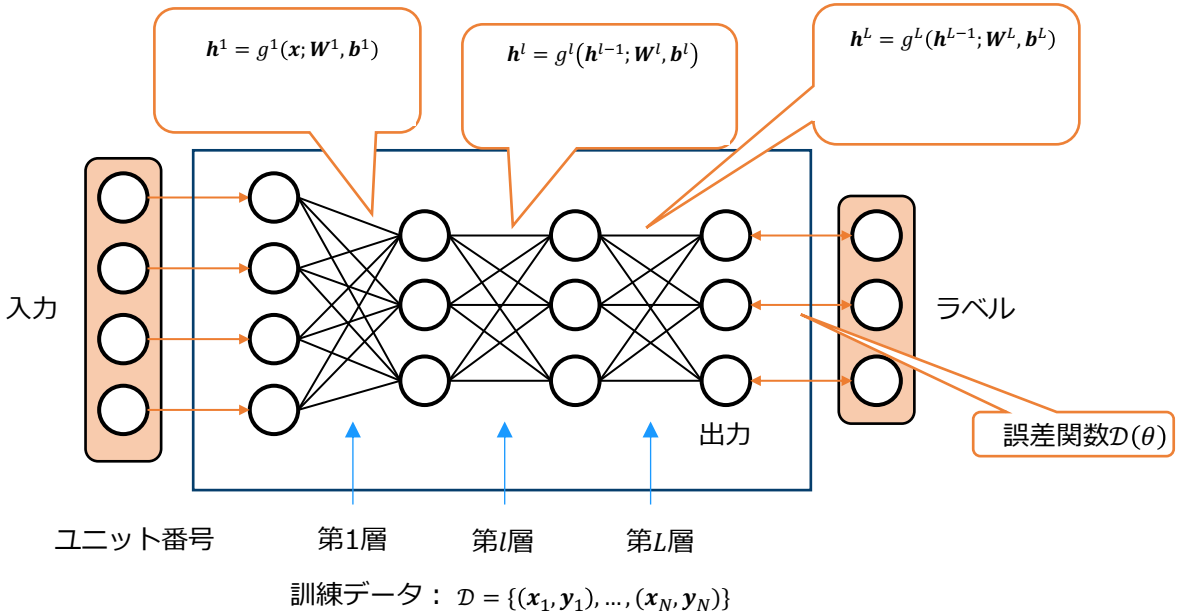


詳細は補足を参照

©Yutaka Matsuo. All rights reserved.

ネットワークの表記

- 層の数：L（右図は3層と数える）
- $\boldsymbol{u}_j^l$： l層目，j番目の出力（活性化関数前）
- g^l： l層目の活性化関数
- $\boldsymbol{h}_j^l$： l層目，j番目の出力（活性化関数後）
- $\boldsymbol{W}^l$：l層目の重み
- $\boldsymbol{b}^l$：l層目のバイアス
- $\boldsymbol{h}^0 = \boldsymbol{x}$, $\boldsymbol{h}^L = \hat{\boldsymbol{y}}$（最初は入力 $\boldsymbol{x}$，最後は出力 $\hat{\boldsymbol{y}}$）
- $\boldsymbol{y}$ は真の出力（ラベル）



$$\boldsymbol{h}^l = g^l\left(\boldsymbol{u}^l\right) = g^l\left((\boldsymbol{W}^l)^T \boldsymbol{h}^{l-1} + \boldsymbol{b}^l\right)$$

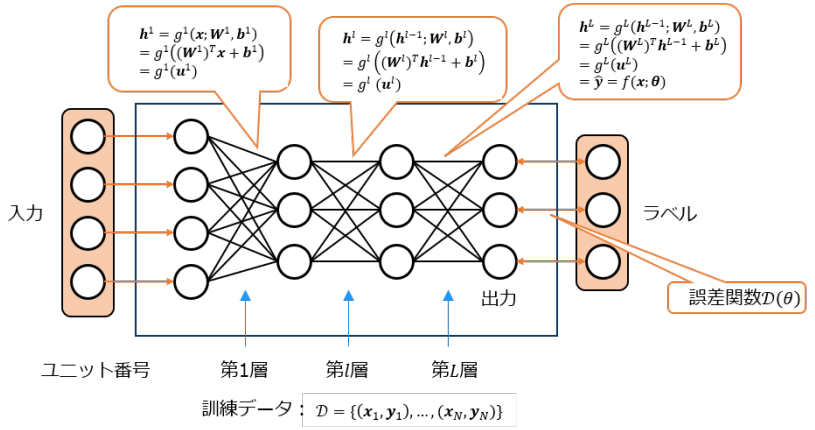
©Yutaka Matsuo. All rights reserved.

MLPの順伝播の擬似コード

input $x, W = \{W^1 \cdots W^L\}, b = \{b^1 \cdots b^L\}$
output $\hat{y}$

1. set $h^0 = x$ # 変数 h^0 に入力 x を代入
2. for $l = 1, \ldots, L$ do # 第1層から第L層までの計算を繰り返し行う
3. $u^l = (W^l)^T h^{l-1} + b^l$ # 活性化関数前の計算を行う
4. $h^l = g^l(u^l)$ # 活性化関数 g^l を通す
5. end for
6. return $\hat{y} = h^L$ # h^L の結果を出力

$$h^l = g^l(u^l) = g^l\left((W^l)^T h^{l-1} + b^l\right)$$



©Yutaka Matsuo. All rights reserved.

表記に関する補足

- 層の単位・数え方
  - [入力→隠れ層→出力層]を3層と呼ぶ流儀（こちらが元々）と
    2層と呼ぶ流儀（最近の流れ）
  - 本講義では2層と呼ぶ方式で統一
    - 一つのパーセプトロン（関数・変換）が一つの層に対応する
    - 実装上はこちらの方が自然

- ユニットは**ニューロン（Neuron）**と呼ばれることもある
  - 深層学習ではユニットの方がよく聞く場合もある

- 誤差関数は損失関数，コスト関数，目的関数と呼ばれることもある
  - これらは微妙に意味が異なるが，同じものとみなされることも多い

©Yutaka Matsuo. All rights reserved.

ここまでのまとめ

- 非線形なデータを分離するための基本的な戦略は，線形分離可能な空間に写像する関数 ϕ を利用すること
  - 写像関数を獲得する方法の一つとして，
    写像関数自体もデータから学習することが挙げられる

- MLPでは隠れ層および活性化関数を利用することで非線形な関数を近似
  - 隠れ層，活性化関数
  - 順伝播の式

©Yutaka Matsuo. All rights reserved.

目次

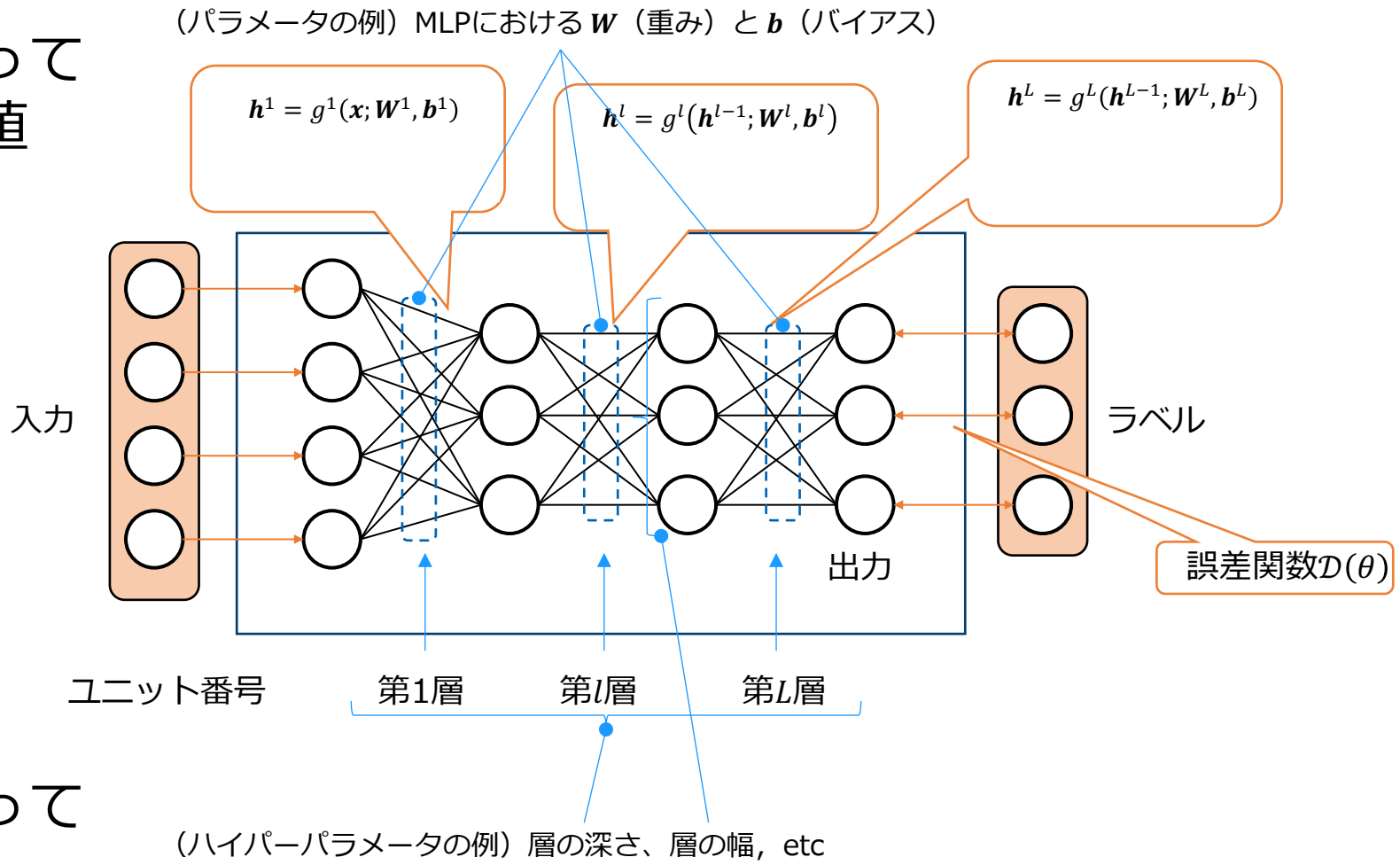
- 前回までのおさらいと非線形問題
- MLP（多層パーセプトロン）の基本的な考え方と構成要素
- **MLPの設定項目（ハイパーパラメータ・アーキテクチャ・誤差関数）**
- 誤差逆伝播法と微分の連鎖律
- 多層化する際の問題点

©Yutaka Matsuo. All rights reserved.

パラメータとハイパーパラメータ

パラメータ

学習アルゴリズムによって
（自動で）調整される値



ハイパーパラメータ

学習アルゴリズムによって
調整しない/できない値

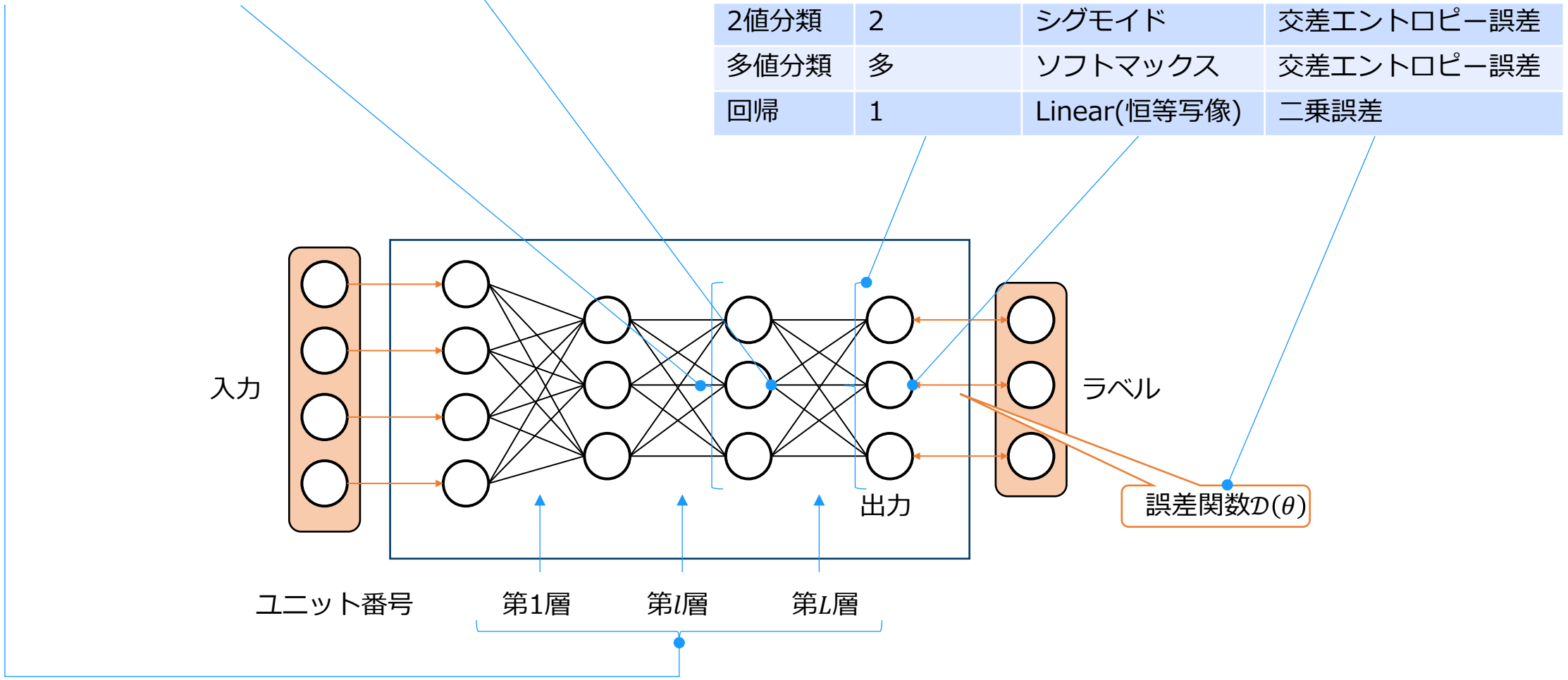
©Yutaka Matsuo. All rights reserved.

MLPを使う上で設定する項目

隠れ層の設計（ハイパーパラメータ）
層の数，各層のユニット数，活性化関数など

タスクに合わせて出力層、誤差関数を設定

	ユニット数	活性化関数	誤差関数
2値分類	2	シグモイド	交差エントロピー誤差
多値分類	多	ソフトマックス	交差エントロピー誤差
回帰	1	Linear(恒等写像)	二乗誤差



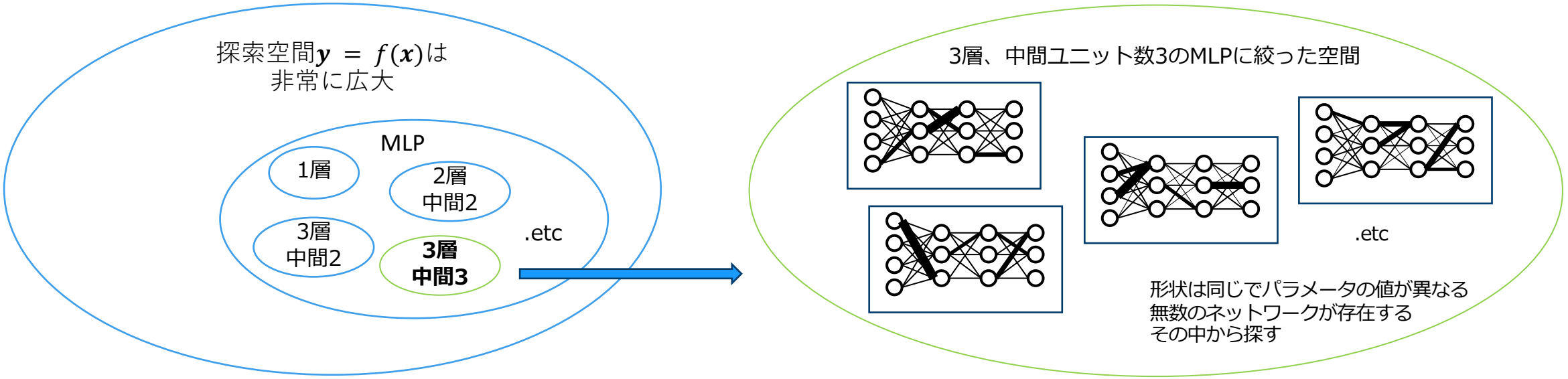
©Yutaka Matsuo. All rights reserved.

モデル探索の側面から見たMLPの設定項目

- 学習とは入出力$(\boldsymbol{x}, \boldsymbol{y})$ の関係 $\boldsymbol{y} = f(\boldsymbol{x})$ をよりよく表せる関数 f の探索である
- 探索空間を有限にするために，モデルをパラメトリック(*)に記述して**探索対象とする関数の空間(範囲)**を限定し，パラメータを調整することで探索を行う

① 探索空間を限定
まずハイパーパラメータを設定することによって
モデル f_θを選択し、探索空間を狭める

② パラメータを調整
モデル f_θを選択した後は，最も妥当な
パラメータ $\boldsymbol{\theta}$ をデータに基づいて探す（学習）



関係 $\boldsymbol{y} = f(\boldsymbol{x})$ をよく表せる関数が見つかり
そうなモデル f_θ を選択することが重要
MLPの場合は，タスクに合わせた出力層と
隠れ層の設計を行う

(*) パラメータによって記述されるモデルをパラメトリックモデル（Parametric Model）と呼ぶ
パラメータを用いて記述されないモデルはノンパラメトリックモデル（Nonparametric Model）と呼ぶ

©Yutaka Matsuo. All rights reserved.

目次

- 前回までのおさらいと非線形問題
- MLP（多層パーセプトロン）の基本的な考え方と構成要素
- MLPの設定項目（ハイパーパラメータ・アーキテクチャ・誤差関数）
- **誤差逆伝播法と微分の連鎖律**
- 多層化する際の問題点

©Yutaka Matsuo. All rights reserved.

最適化の一般的な方針

- 閉形式で解ける場合 → 解析的に計算して解を求める
  - 例：線形回帰（正規方程式を解けば良い）
  - 閉形式：四則演算や初等関数の合成関数による解の表現
  - 線形回帰 $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{W}$ の場合（バイアス項も $\boldsymbol{X}, \boldsymbol{W}$ に入れている），
    最適な $\boldsymbol{W} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y}$ と計算できる（=正規方程式）

- 閉形式で解くのが困難な場合
  → 勾配降下法などの勾配法（反復解法）を用いる
  - 例：ロジスティック回帰，MLP
  - 勾配降下法以外では，2次勾配を用いたニュートン法など

©Yutaka Matsuo. All rights reserved.

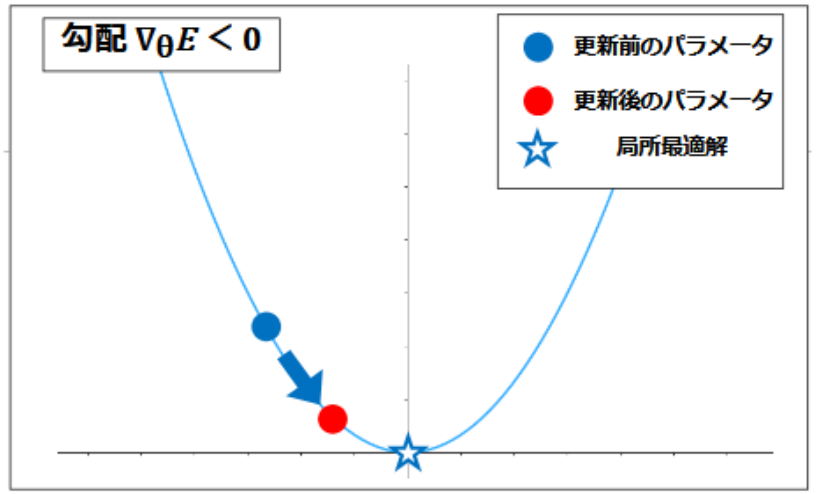
勾配降下法（おさらい）

1. 全データの誤差関数 $E(\boldsymbol{\theta})$ のパラメータ $\boldsymbol{\theta}$ における勾配 $\nabla E = \frac{\partial E(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$ を計算
2. 次の更新則をパラメータに適用

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla E\left(\boldsymbol{\theta}^{(t)}\right)$$

ηは学習率（1回の更新あたりのパラメータの変更度合いを決める）

3. 勾配が0になるまで2に戻って繰り返す



- 誤差関数：例えば回帰なら出力と正解の最小二乗誤差（MSE）
- パラメータの値を勾配の向きとは逆向きに進めることで，
  誤差関数を最小化するような値に近づける

©Yutaka Matsuo. All rights reserved.

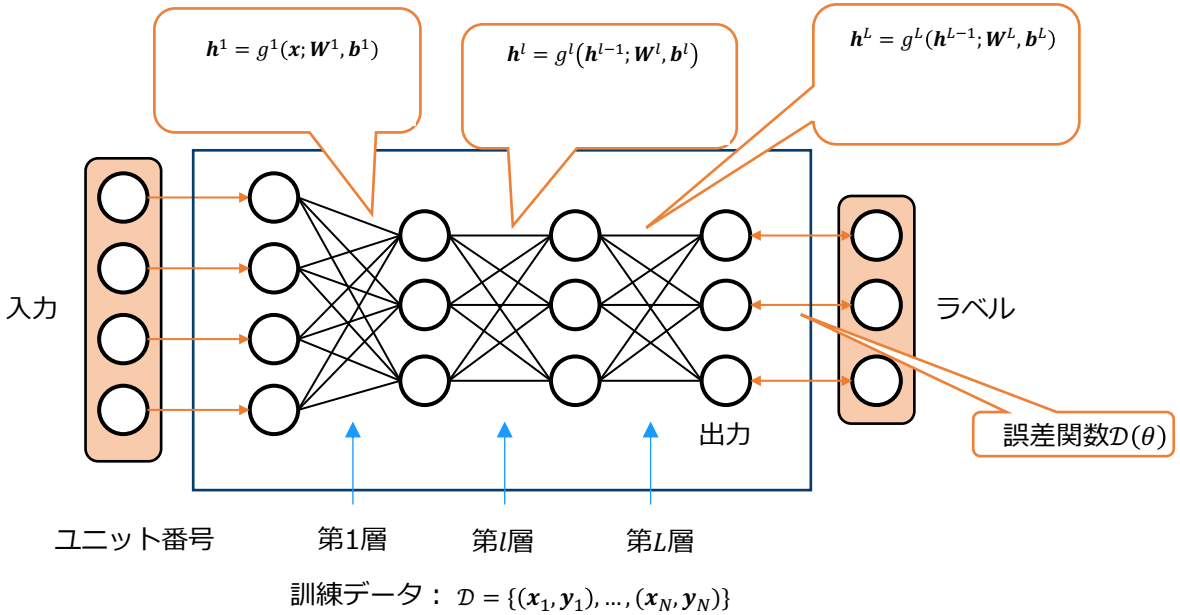
ミニバッチ勾配降下法（おさらい）

- データ集合全て（バッチ）を使って学習するのではなく，
  一部のサンプル（ミニバッチ）を毎回ランダムに選択して学習する方法
  - データ集合から，サンプルを「確率的」に選択して誤差関数を求める
  - 一回のサンプルあたりに含まれるデータの数をバッチサイズという
    - バッチサイズを1としたミニバッチ勾配降下法を
      確率的勾配降下法（Stochastic Gradient Descent; SGD）という
  - 確率的にデータを選択することで，最適化が局所解に固定されにくくなる
  - 全データ集合を使って学習しても収束してない場合は，もう何周か回してみる
  - データ集合を1周回す単位をエポックという
    - 深層学習だと基本的に複数エポック必要
    - エポックごとにデータをシャッフルし，分割して順番に使用する

©Yutaka Matsuo. All rights reserved.

ネットワークの表記（再掲）

- 層の数：L（右図は3層と数える）
- $\boldsymbol{u}_j^l$： l層目，j番目の出力（活性化関数前）
- g^l： l層目の活性化関数
- $\boldsymbol{h}_j^l$： l層目，j番目の出力（活性化関数後）
- $\boldsymbol{W}^l$：l層目の重み
- $\boldsymbol{b}^l$：l層目のバイアス
- $\boldsymbol{h}^0 = \boldsymbol{x}$, $\boldsymbol{h}^L = \hat{\boldsymbol{y}}$（最初は入力 $\boldsymbol{x}$，最後は出力 $\hat{\boldsymbol{y}}$）
- $\boldsymbol{y}$ は真の出力（ラベル）



$$\boldsymbol{h}^l = g^l(\boldsymbol{u}^l) = g^l\left((\boldsymbol{W}^l)^T \boldsymbol{h}^{l-1} + \boldsymbol{b}^l\right)$$

©Yutaka Matsuo. All rights reserved.

最終層の重みに関する偏微分

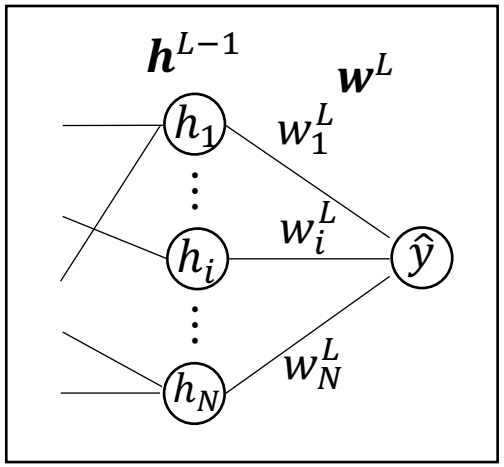
- $\hat{y} = (\boldsymbol{w}^L)^T \boldsymbol{h}^{L-1}$ とする（バイアス項を省いた表記）
  - 誤差関数を $E(\boldsymbol{\theta}) = \frac{1}{2}(\hat{y} - y)^2$ とする（※ 任意の誤差関数で良い）

$$\frac{\partial E(\boldsymbol{\theta})}{\partial w_i^L} = \frac{\partial E(\boldsymbol{\theta})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_i^L}$$

$$\frac{\partial E(\boldsymbol{\theta})}{\partial \hat{y}} = (\hat{y} - y)$$

出力の「誤差」

$L-1$層目の出力



※誤差関数が交差エントロピーの場合も同じ値

©Yutaka Matsuo. All rights reserved.

隠れ層の重みに関する偏微分は？

- 入力に近い層の重みほど深いネストの中に存在してしまう
- 各層について合成関数の微分を毎回計算するのは効率が良くない

$$\frac{\partial E(\boldsymbol{\theta})}{\partial w_{i,j}^{l}} = \frac{\partial E(\boldsymbol{\theta})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \boldsymbol{h}^{L-1}} \frac{\partial \boldsymbol{h}^{L-1}}{\partial \boldsymbol{u}^{L-1}} \frac{\partial \boldsymbol{u}^{L-1}}{\partial \boldsymbol{h}^{L-2}} \cdots \frac{\partial u_{j}^{l}}{\partial w_{i,j}^{l}}$$

各層 l に関してこれを知りたい

※ $\boldsymbol{u}^{l} = (\boldsymbol{W}^{l})^{T} \boldsymbol{h}^{l-1}$ とする

©Yutaka Matsuo. All rights reserved.

補足：合成関数の微分（連鎖律）

- 二変数関数: $f(x, y)$

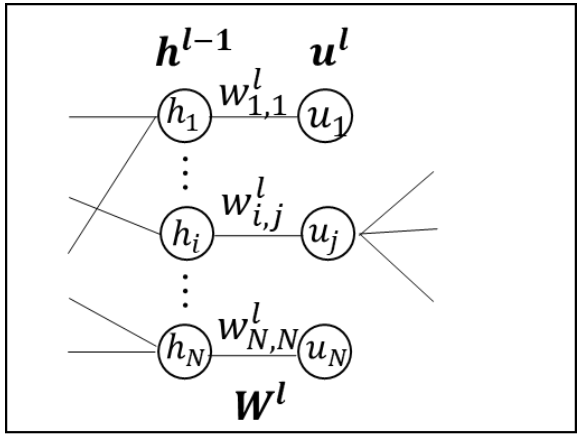
- 多変数関数: $x(t_1, \ldots, t_k, \ldots, t_K)$, $y(t_1, \ldots, t_k, \ldots, t_K)$ とする

$$\frac{df(x,y)}{dt_k} = \frac{df}{dx}\frac{dx}{dt_k} + \frac{df}{dy}\frac{dy}{dt_k}$$

©Yutaka Matsuo. All rights reserved.

隠れ層の重みに関する偏微分

$$\frac{\partial E(\boldsymbol{\theta})}{\partial w_{i,j}^{l}} = \frac{\partial E(\boldsymbol{\theta})}{\partial u_{j}^{l}} \frac{\partial u_{j}^{l}}{\partial w_{i,j}^{l}}$$



$$\frac{\partial E(\boldsymbol{\theta})}{\partial u_{j}^{l}} = \sum_{k} \frac{\partial E(\boldsymbol{\theta})}{\partial u_{k}^{l+1}} \frac{\partial u_{k}^{l+1}}{\partial u_{j}^{l}}$$

$$\frac{\partial u_{j}^{l}}{\partial w_{i,j}^{l}} = h_{i}^{l-1}$$

l層目の「誤差」　$l+1$層目の「誤差」

$l-1$層目の出力

※ k は l 層 j 番目と結合がある上層のユニット

©Yutaka Matsuo. All rights reserved.

誤差逆伝播法

- 層lの「誤差」 $\frac{\partial E(\boldsymbol{\theta})}{\partial u_j^l}$ を δ_j^l と表すと，前ページの式より

$$\delta_j^l = \sum_k \delta_k^{l+1} \frac{\partial u_k^{l+1}}{\partial u_j^l} \rightarrow \text{層 } l \text{ の誤差と } l+1 \text{ の誤差の関係}$$

- また $u_k^{l+1} = \sum_j w_{j,k}^{l+1} h_j^l = \sum_j w_{j,k}^{l+1} g(u_j^l)$（順伝播）から，$\frac{\partial u_k^{l+1}}{\partial u_j^l} = w_{j,k}^{l+1} g'(u_j^l)$ となる

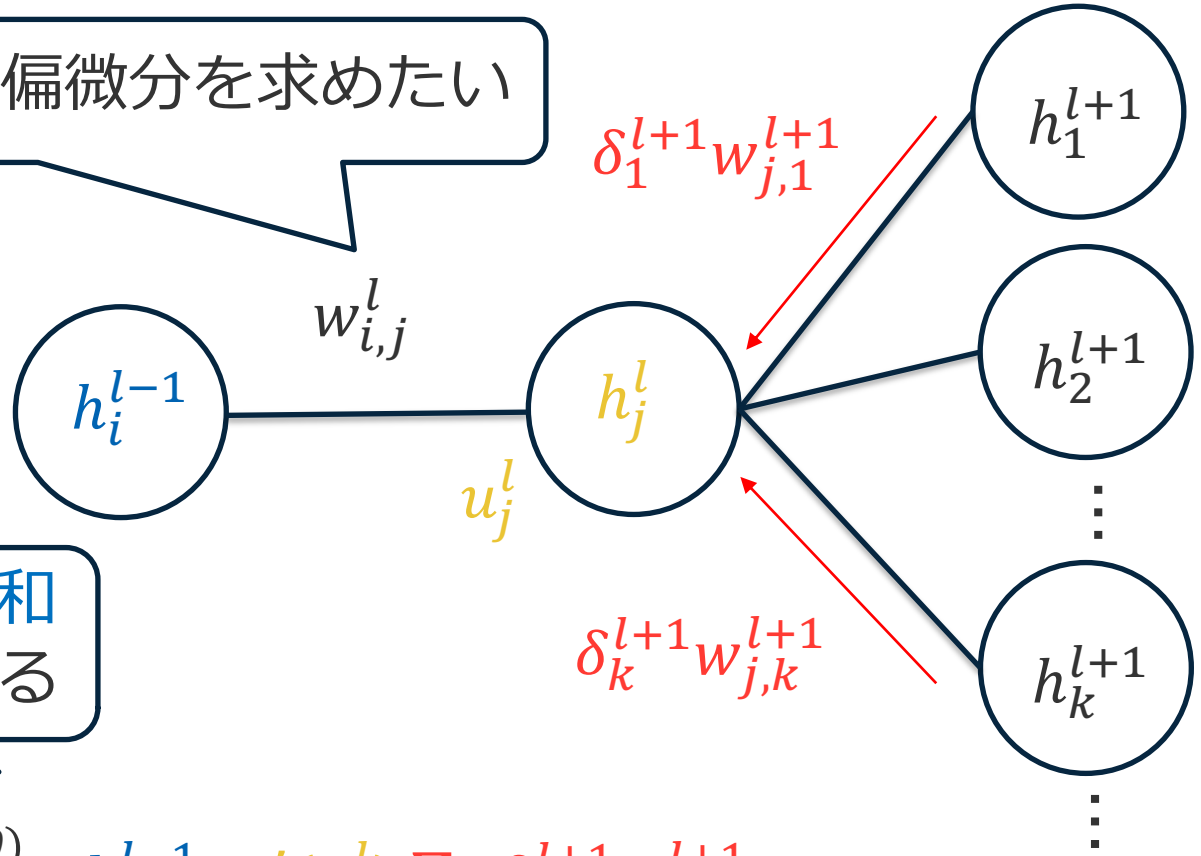
- 上の式より，自分より上層の誤差が分かれば簡単に誤差が計算できる
  - よって，最終層から誤差を求めて下層に伝播していけば，効率的に全パラメータの勾配を計算できる

➡ **誤差逆伝播法（Backpropagation）**

©Yutaka Matsuo. All rights reserved.

誤差逆伝播法の直感的理解

Wについて誤差関数の偏微分を求めたい



上層での誤差の重み付き和
とユニットi, jの値で決まる

$$\frac{\partial E(\boldsymbol{\theta})}{\partial w_{i,j}^l} = h_i^{l-1}\left(g'(u_j^l) \sum_k \delta_k^{l+1} w_{j,k}^{l+1} \right)$$

©Yutaka Matsuo. All rights reserved.

誤差逆伝播法

- 誤差逆伝播の擬似コード
  - ⊙はアダマール積（ベクトル表記になっているが，ここまでと同じことを言っている）

1. **# 逆伝播**
2. set $\boldsymbol{\delta} = \nabla_{\hat{\boldsymbol{y}}} E(\boldsymbol{\theta}) = \hat{\boldsymbol{y}} - \boldsymbol{y}$ **# 出力層の誤差（$\hat{y} - y$）**
3. for $l = L - 1, \ldots, 1$ do **# 最終層は2.で求めているため $L-1$ から計算**
4. $\quad \boldsymbol{\delta} = \nabla_{\boldsymbol{u}^l} E(\boldsymbol{\theta}) = g'(\boldsymbol{u}^l) \odot (\boldsymbol{W}^{l+1}\boldsymbol{\delta})$ **# 各層の誤差**
5. $\quad \nabla_{\boldsymbol{W}^l} E(\theta) = \boldsymbol{h}^{l-1}\boldsymbol{\delta}^T$ **# 誤差を逆伝播**
6. end for

©Yutaka Matsuo. All rights reserved.

誤差逆伝播法の歴史的背景

- 誤差逆伝播法はRumelhartらによって1980年代に「再発見」された [Rumelhart+ 1986]
  - 実は同等の方法は以前にも提案されていた [Widrow 1960] [Amari 1967]

- 従来はcredit assignment問題によって，多層のパーセプトロンは学習できないとされていた
  - 誤差逆伝播法によって2層のパーセプトロンでも学習できることが示された
  - 第2次ニューラルネットワークブーム

- ただし3層以上では適切に学習できないことがわかり，このブームは収束
  - 2006年に，深層ニューラルネットワークが学習できることが示されるまで，ニューラルネットワークは冬の時代を迎えた
  - 機械学習自体の研究は続けられ，サポートベクトルマシン（SVM）[Cortes+ 1995]など素晴らしい成果が生まれた

©Yutaka Matsuo. All rights reserved.

ここまでのまとめ

- 機械学習アルゴリズムの最適化
  - 閉形式で解ける場合 → 解く
  - 閉形式で解けない場合 → 勾配法など
    - MLPは閉形式で解けないので勾配法を通常使う
- 確率的勾配降下法
  - 学習にランダム性を追加することで局所解に陥ることを防ぐ
- 誤差逆伝播法
  - MLPの各層のパラメータの勾配を効率的に計算する方法
  - 第2次ニューラルネットワークブームへと導いた

©Yutaka Matsuo. All rights reserved.

目次

- 前回までのおさらいと非線形問題
- MLP（多層パーセプトロン）の基本的な考え方と構成要素
- MLPの設定項目（ハイパーパラメータ・アーキテクチャ・誤差関数）
- 誤差逆伝播法と微分の連鎖律
- **多層化する際の問題点**

©Yutaka Matsuo. All rights reserved.

多層化が難しかった理由

- MLPは3層以上の場合，適切に学習できなかった
  - ブームが収束し冬の時代に
  - 多層化や幅（ユニット数）の拡張でより複雑な関数を表現可（モデルの容量が大きい）

- 多層化が難しい理由
  - 勾配消失・勾配爆発問題
  - 計算量の問題
  - 高次元空間の探索の難しさ

→ 各問題の詳細と，どのように解決したかを説明していく

©Yutaka Matsuo. All rights reserved.

勾配消失問題・勾配爆発問題

- 誤差逆伝播法では上層の誤差を利用し下層の誤差を計算していた
  - 上層の誤差 δ が発散したり0になってしまうと，下層に誤差情報が適切に伝わらなくなる

$$\delta_j^l = \sum_k \delta_k^{l+1} \frac{\partial u_k^{l+1}}{\partial u_j^l}$$

誤差 0 → 勾配消失
発散　 → 勾配爆発

  - 順伝搬は活性化関数で値域が制限されるが，逆伝播では制約がない

©Yutaka Matsuo. All rights reserved.

勾配消失問題

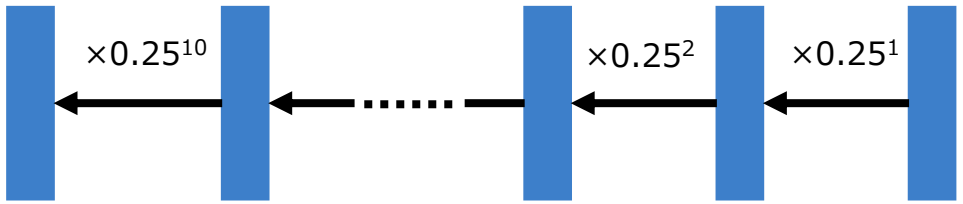
- 逆伝播の際に,各層で活性化関数の微分が乗算される
  - シグモイド関数を活性化関数とした場合,微分すると値は0から0.25の範囲になる

層が増えた際に小さな値の乗算を繰り返すことになり
入力層近くの誤差が0に近づいてしまう（更新できない）

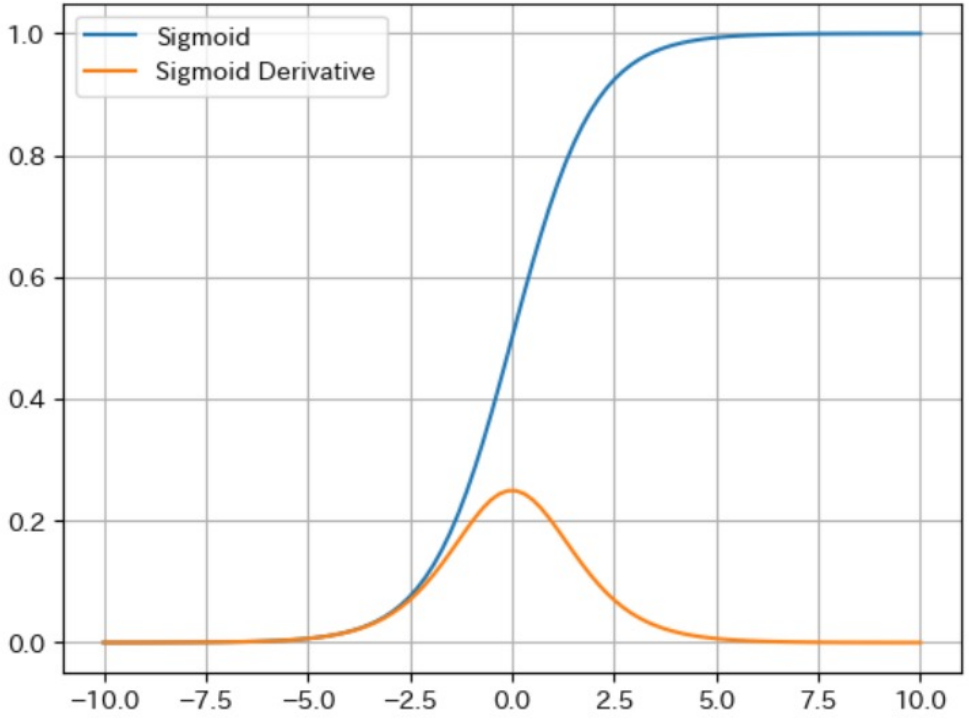
$$\delta_j^l = g'(u_j^l) \sum_k \delta_k^{l+1} w_{j,k}^{l+1}$$

勾配消失問題
（Vanishing Gradient Problem）

シグモイド関数を用いた10層のMLPの場合



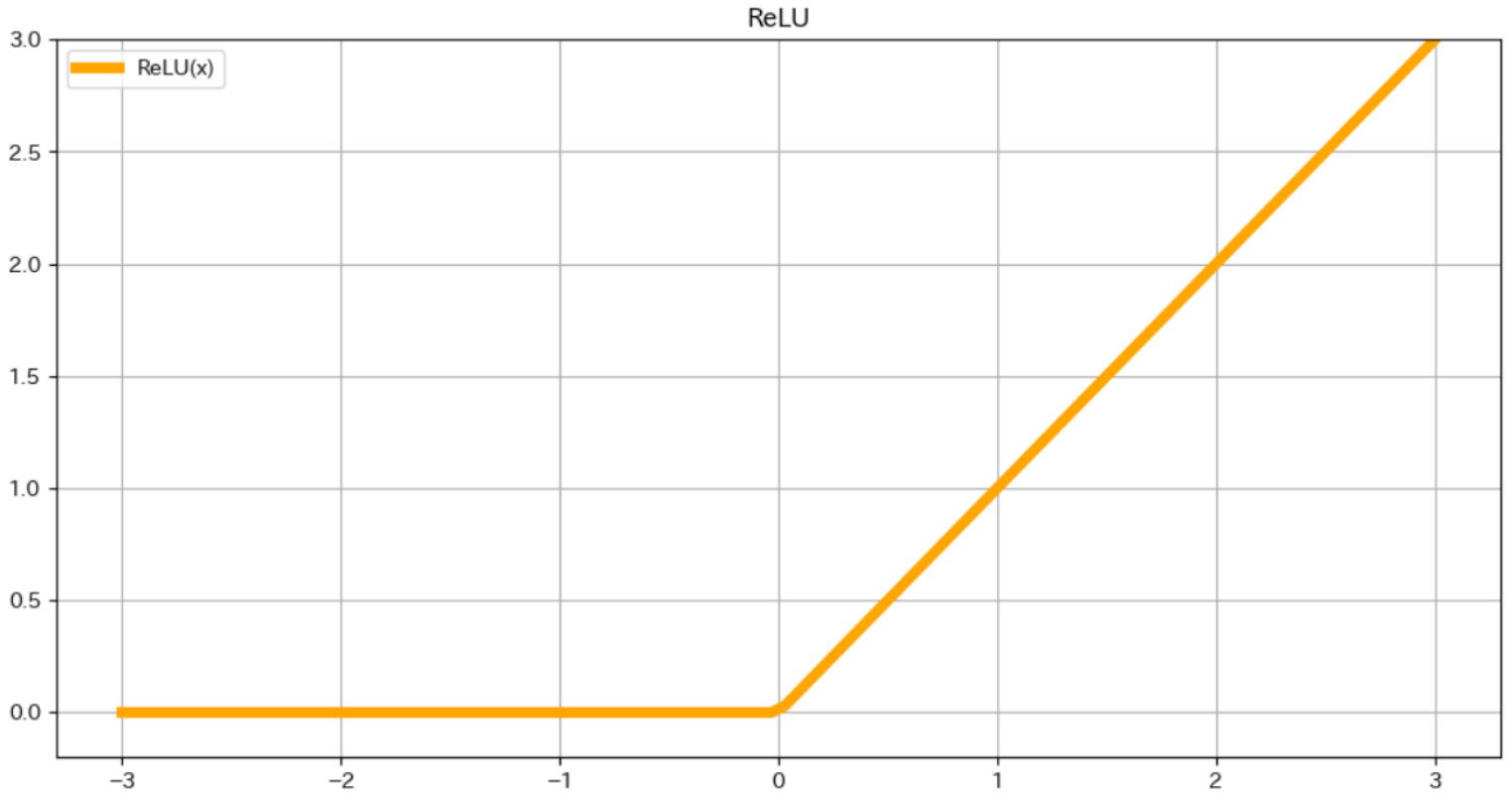
$$0.25^{10} = 0.0000009537 \approx 0$$



©Yutaka Matsuo. All rights reserved.

勾配消失問題

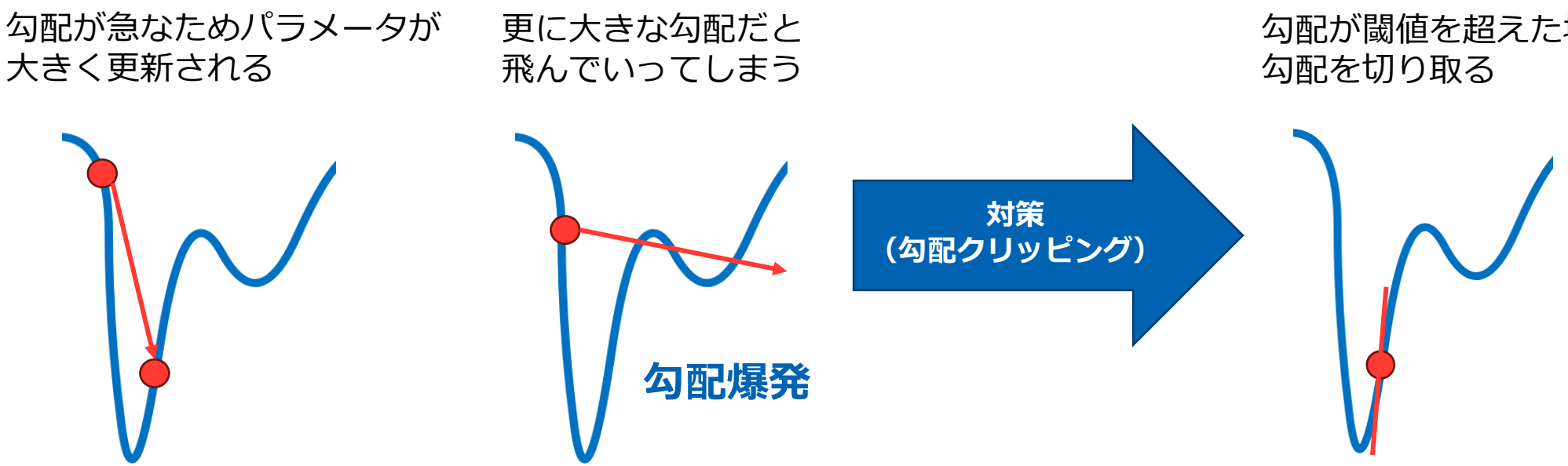
- 解決方法としてReLUを活性化関数として利用
  - 非線形性を持ちつつ，入力が正のとき微分値が常に1，負のときは0となる
    現在のニューラルネットワークではこれが主流



©Yutaka Matsuo. All rights reserved.

勾配爆発問題

- 多層のMLPではパラメータの組み合わせが複雑であり，崖のような急な勾配が発生してしまう場合がある（下図参照）
  - この場合，大きな勾配によって適切なパラメータ領域から飛んでいってしまう恐れがある
    → **勾配爆発問題（Exploding Gradient Problem）**
- **勾配クリッピング（Gradient Clipping）** [Pascanu+ 2012]
  - 勾配爆発の対策として，勾配の大きさを制限する方法が用いられる



©Yutaka Matsuo. All rights reserved.

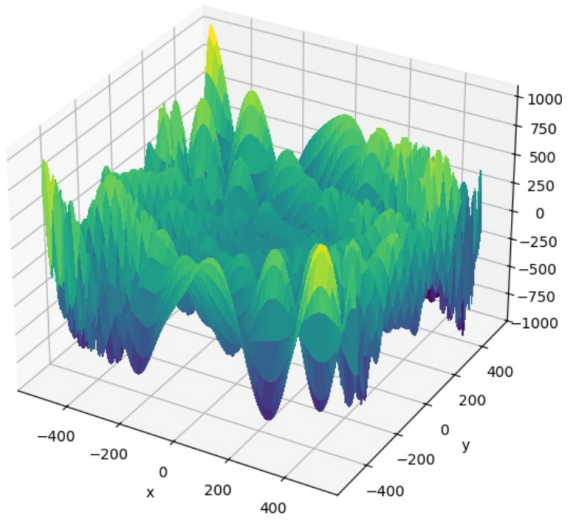
高次元空間の探索の難しさ

- 勾配降下法で最適化を行う上で，
  ニューラルネットワークの高次元なパラメータ空間は非常に複雑なため，
  グローバルな最小値を探し出すのはとても難しい

  図は最適化のベンチマークに用いられる関数の例

  - このような複雑な関数に対し，
    「ある一点（足元）の傾きだけを元に最小値を探す」
    というイメージ

  - 深層ニューラルネットワークの場合は，数万次元にもなる
    → パラメータの更新方法を工夫する必要がある



最適化アルゴリズムのベンチマーク
に用いられる非線形関数の例

©Yutaka Matsuo. All rights reserved.

第2次ニューラルネットワークブームの終焉と復活

- 第2次ニューラルネットワークブーム
  - Rumelhart, Hinton & Williams（1986）が誤差逆伝播法による2層MLPの学習に成功
    - 活性化関数として微分可能なシグモイドを提案
  - しかし，勾配消失・爆発や過学習の問題を解決することができなかった
    → 3層以上への拡張が困難

- 第3次ニューラルネットワークブーム
  - ReLUや勾配クリッピングなどの工夫，またさらなる深層化技術（バッチ正則化やスキップ接続など）の開発によってニューラルネットワークブームが復活
  - Hintonが2006に発表した自己符号化を一層ずつ逐次的に行う事前学習方法も，このブームのきっかけになった

©Yutaka Matsuo. All rights reserved.

まとめ

- 線形分離可能性
  - 線形回帰・ロジスティック回帰
  - 単純パーセプトロン

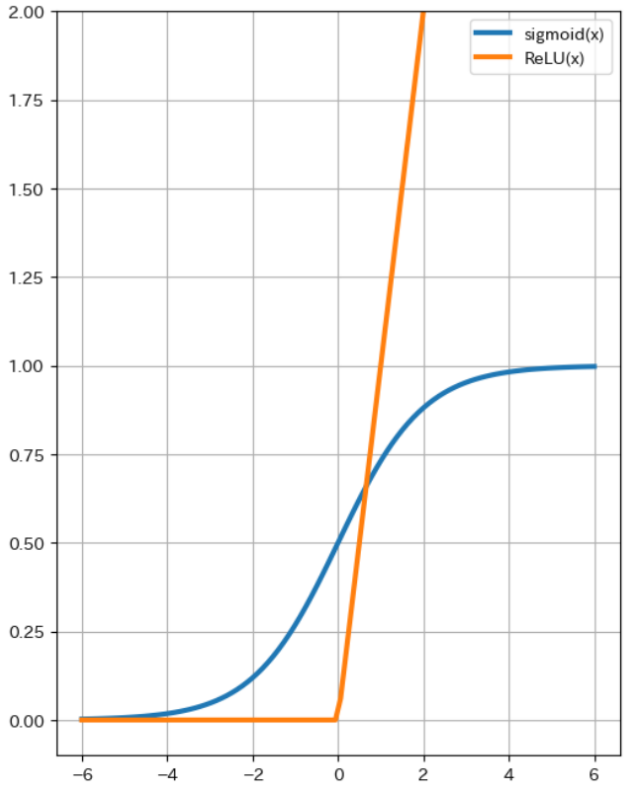
- MLP
  - 各種構成要素
  - 誤差逆伝播法
  - 確率的勾配降下法
  - 多層化のための工夫

©Yutaka Matsuo. All rights reserved.

補足：ReLU

- Rectified Linear Unit（ReLU）
  - 入力値が正の場合はその値を出力，負の場合は0を出力する関数

$$h_{\mathrm{ReLU}}(x) = \begin{cases} x\ (x > 0) \\ 0\ (x \leq 0) \end{cases} \quad h'_{\mathrm{ReLU}}(x) = \begin{cases} 1\ (x > 0) \\ 0\ (x \leq 0) \end{cases}$$

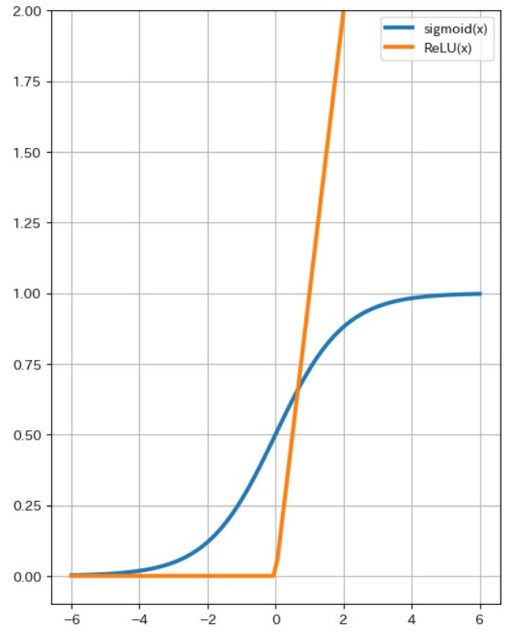


©Yutaka Matsuo. All rights reserved.

補足：ReLU

- Rectified Linear Unit（ReLU）
  - 入力値が正の場合はその値を出力，負の場合は0を出力する関数

$$h_{\mathrm{ReLU}}(x) = \begin{cases} x\ (x > 0) \\ 0\ (x \leq 0) \end{cases} \quad h'_{\mathrm{ReLU}}(x) = \begin{cases} 1\ (x > 0) \\ 0\ (x \leq 0) \end{cases}$$



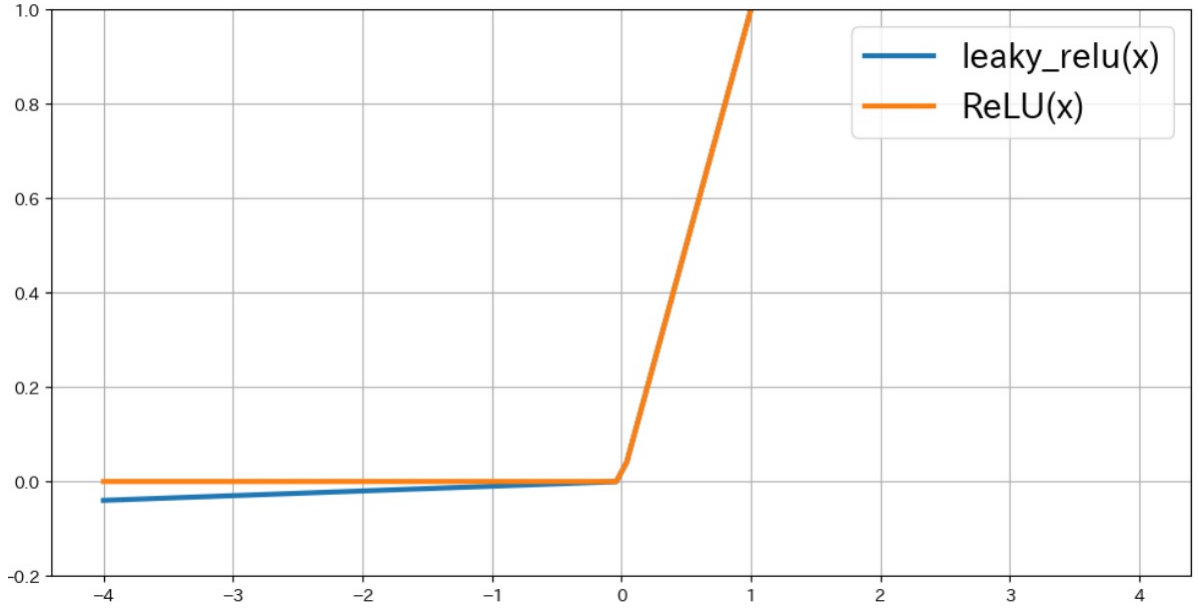
- **特徴**
  - CNN（第5回を参照）やDNNで一般的に用いられる
  - 計算式がシンプルであり，処理が早い
  - ニューラルネットワークにおける勾配消失問題を解消できる

©Yutaka Matsuo. All rights reserved.

補足：LReLU

- Leaky Rectified Linear Unit（LReLU） [Mass+ 2013]
  - ReLUを基にした関数
  - ReLUとは異なり，入力値が負の場合は定数a倍（$a << 1$）をかけたaxを返す関数
    $a = 0.01$に設定することが一般的である

$$h_{\mathrm{LReLU}}(x) = \begin{cases} x\ (x > 0) \\ ax\ (x \leq 0, a \ll 1) \end{cases} \qquad h'_{\mathrm{LReLU}}(x) = \begin{cases} 1\ (x > 0) \\ a\ (x \leq 0, a \ll 1) \end{cases}$$



$a = 0.01$

©Yutaka Matsuo. All rights reserved.

補足：LReLU

- Leaky Rectified Linear Unit（LReLU）[Mass+ 2013]
  - ReLUを基にした関数
  - ReLUとは異なり，入力値が負の場合は定数a倍（$a << 1$）をかけたaxを返す関数
    $a = 0.01$に設定することが一般的である

$$h_{\mathrm{LReLU}}(x) = \begin{cases} x\ (x > 0) \\ ax\ (x \leq 0, a \ll 1) \end{cases} \qquad h'_{\mathrm{LReLU}}(x) = \begin{cases} 1\ (x > 0) \\ a\ (x \leq 0, a \ll 1) \end{cases}$$

- **特徴**
  - CNN（第5回を参照）やDNNで一般的に用いられる
  - 入力が0以下であっても勾配が存在するので学習が進みやすい
  - 負の入力値に対しても許容するので，勾配消失問題を解消できる

©Yutaka Matsuo. All rights reserved.

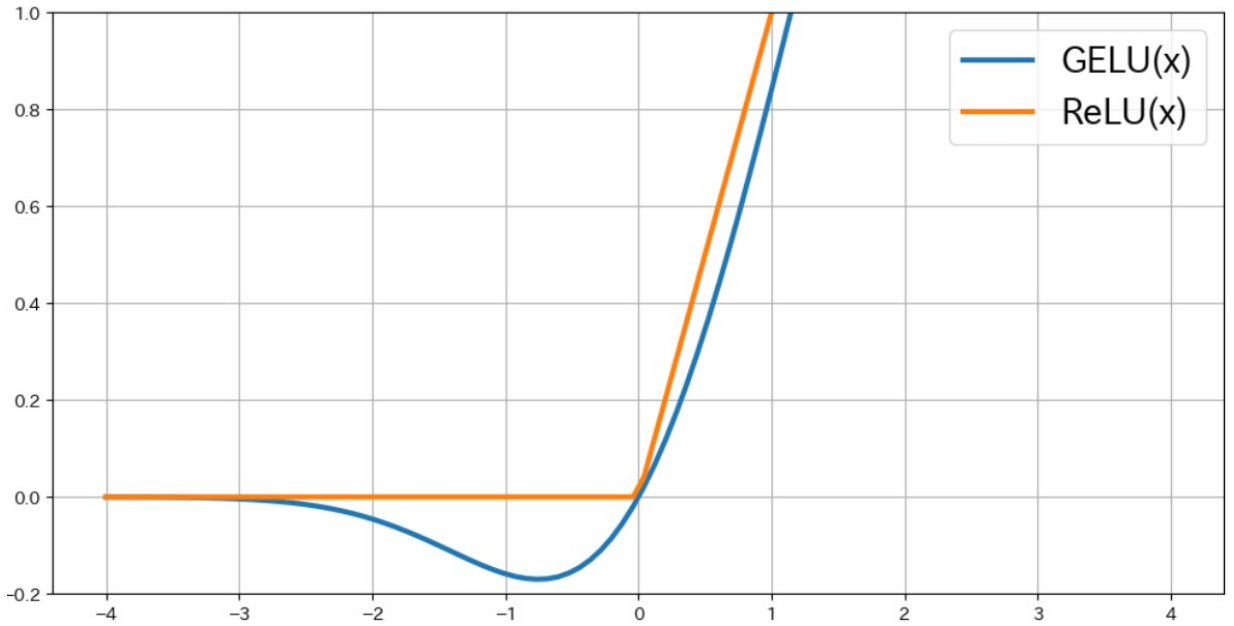
補足：GELU

- **Gaussian Error Linear Unit（GELU）**[Hendrycks & Gimpel 2016]
  - ReLUとは異なり，入力値に対してガウス分布の累計分布関数をかけて出力する関数

$$h_{\mathrm{GELU}}(x) = x\Phi(x)$$

$$h'_{\mathrm{GELU}}(x) = \Phi(x) + x\Phi'(x)$$

- $\Phi(x)$：ガウス分布の累計分布関数



©Yutaka Matsuo. All rights reserved.

補足：GELU

- Gaussian Error Linear Unit ; GELU [Hendrycks & Gimpel 2016]
  - ReLUとは異なり，入力値に対してガウス分布の累計分布関数をかけて出力する関数

$$h_{\mathrm{GELU}}(x) = x\Phi(x)$$

$$h'_{\mathrm{GELU}}(x) = \Phi(x) + x\Phi'(x)$$

- $\Phi(x)$：ガウス分布の累計分布関数

- **特徴**
  - Transformer（第8回を参照）系のような巨大規模モデルで多用される
  - ReLUに確率的要素を取り入れることで，活性化関数をなめらかにする
  - 負の入力値に対しても許容するので，勾配消失問題を解消できる
  - ReLUと比べて式が複雑で，計算量が大きい

©Yutaka Matsuo. All rights reserved.

補足：万能近似定理

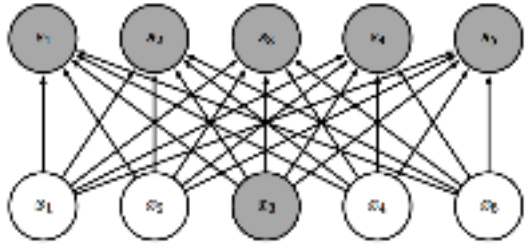
- **万能近似定理（Universal Approximation Theorem）** [Hornik+ 1989]
  - 「ネットワークが十分な数の隠れユニットを保つ場合，線形の出力層と
    『押しつぶすことができる』活性化関数を持つ隠れ層が少なくとも1つ含まれる
    順伝播型ネットワークはどんなボレル可測関数でも任意の精度で近似できる」

- どんな関数を学習するかに関わらず，十分なユニット数があるMLPはそれを表現することができる
  - ただし，実際に学習できるかどうかは保証していないことに注意
  - 複雑な関数を表現する際には，「十分なユニット数」は恐らく膨大な数

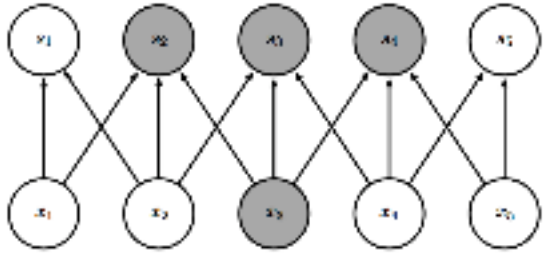
©Yutaka Matsuo. All rights reserved.

補足：その他のアーキテクチャ設計

- **局所結合層（Locally-Connected Layer）**
  - **全結合層（Fully-Connected Layer）**では，
    1つ前の層のユニットの全てと次の層のユニットが
    密結合 → パラメータ数が爆発する可能性
  - 局所結合層では，
    各層のユニットが前の層の一部のユニットとのみ結合
  - データの性質によってパラメータ数を劇的に削減可能
    - 例：画像に対する畳み込み（第5回で説明）

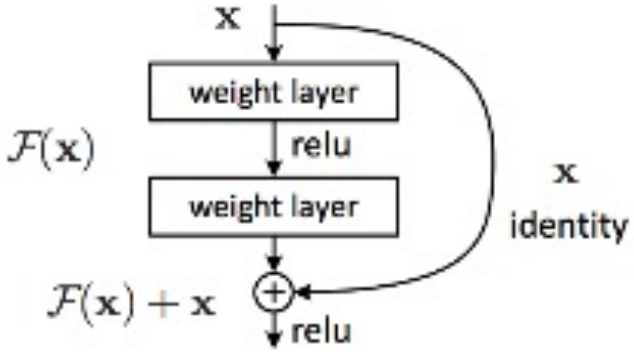


全結合層：密な結合



局所結合層：疎な結合

- **残差接続（Residual Connection）**
  または**スキップ接続（Skip Connection）**[He+ 2015]
  - 第5回で詳しく説明
  - 層間の接続をスキップする
    - より入力の情報が伝わりやすくなる可能性
  - 例：Residual Networks（ResNet）



©Yutaka Matsuo. All rights reserved.

補足：誤差逆伝播法の実装

- 誤差逆伝播は「MLP専用の最適化アルゴリズム」と思われがち
  - 連鎖律による効率的な勾配計算アルゴリズム
  - 任意の関数の勾配計算に利用可能

- 誤差逆伝播法を様々な場面（手法）で実装できるようにしたい

©Yutaka Matsuo. All rights reserved.

補足：誤差逆伝播法の実装

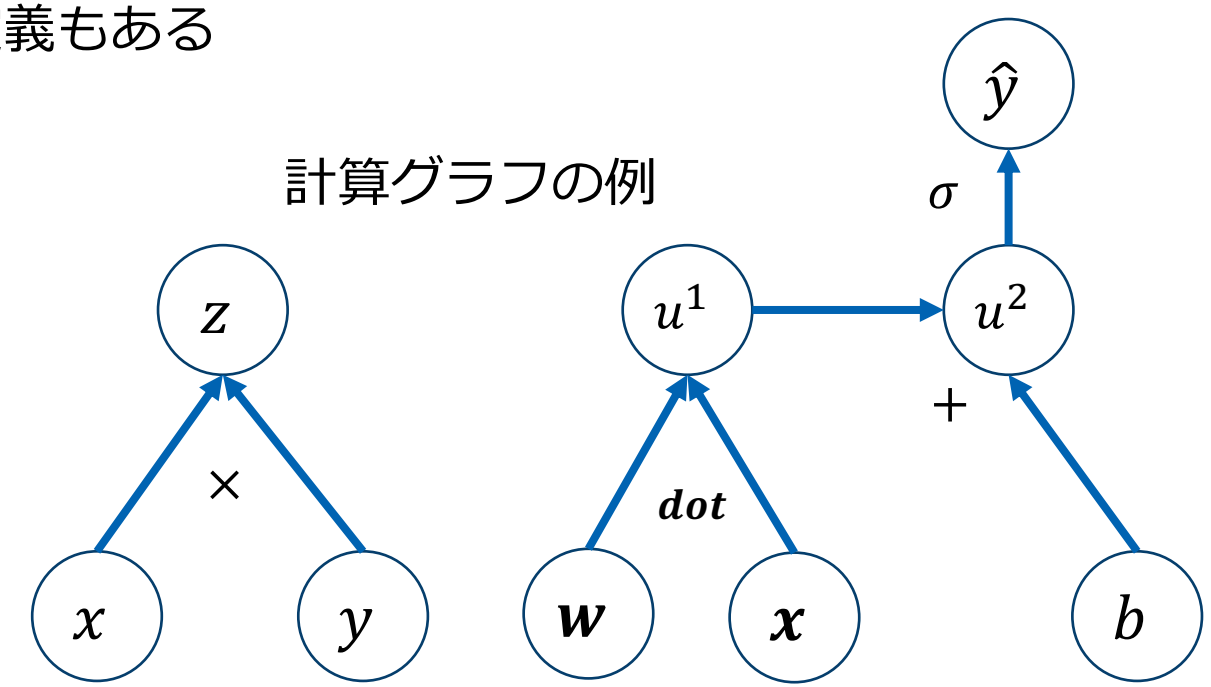
- **計算グラフ（Computational Graph）**
  - 計算をグラフ形式で表現し，視覚的にわかりやすくしたもの
  - ノード（グラフの点）が変数（スカラー，行列，テンソルなど）を表す
  - エッジ（グラフの辺）が演算（変数の関数）を表す

※ノードを演算，エッジを変数とする定義もある

ノード： x y z …

エッジ： × + σ …

計算グラフの例



$z = xy$

$\hat{y} = \sigma(\boldsymbol{w}^T\boldsymbol{x} + b)$

©Yutaka Matsuo. All rights reserved.

補足：誤差逆伝播法の実装

- 計算グラフの勾配の値を求める際には，2種類の数値計算方法がある

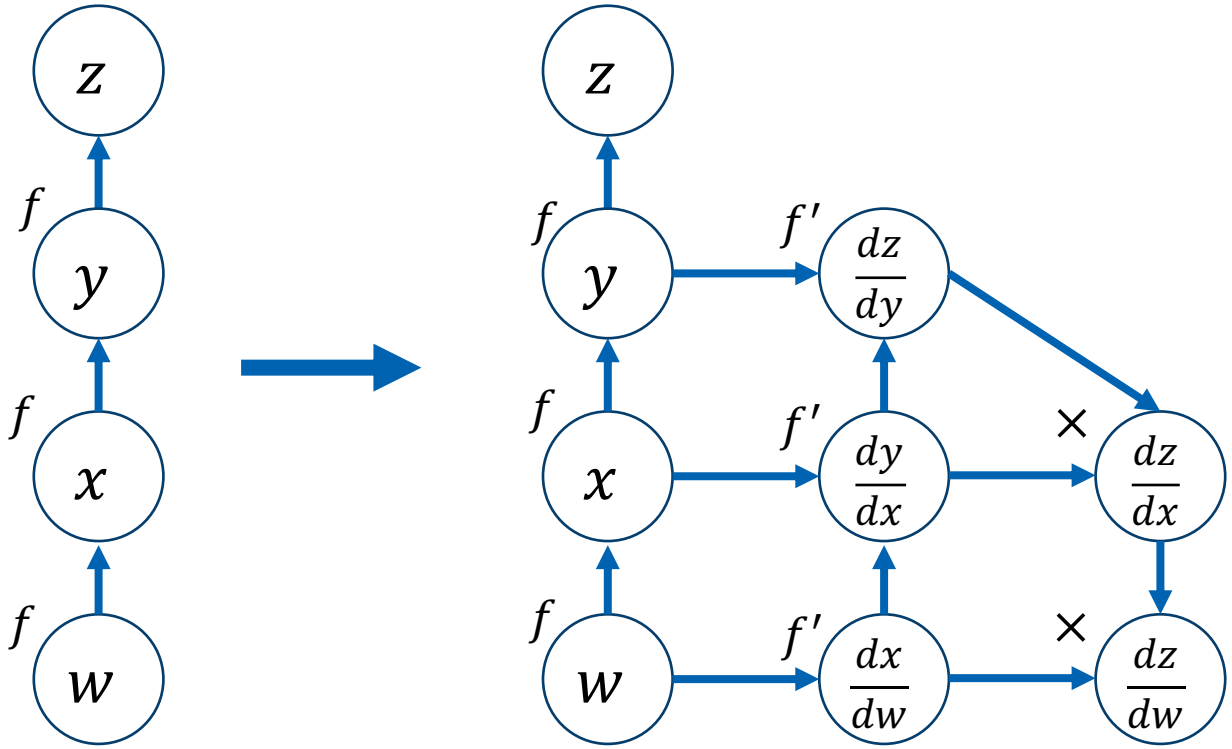
- シンボル-数値間の微分
  - 計算グラフ上の各ノードに対して逆伝播を行い，
    入力に対する勾配を具体的な数値（テンソル）として計算・返す手法
  - 計算グラフは実行中に動的に毎回構築される（define-by-run型）
    柔軟かつ直感的なデバッグが可能
  - PyTorchなどのライブラリで使用されている

- シンボル-シンボル間の微分（次ページで説明）
  - あらかじめ構築した計算グラフに，各演算の微分（勾配）を計算するためのシンボリックなノードを追加しておき，逆伝播時にそのグラフをたどって勾配を求める手法
  - グラフ定義後に実行する（define-and-run型）．最適化や分散処理がしやすい
  - TensorFlowなどのライブラリで使用されている

©Yutaka Matsuo. All rights reserved.

補足：誤差逆伝播法の実装

- 計算グラフで誤差逆伝播を表現
  - 例：$z = f(f(f(w)))$において $\frac{dz}{dw}$ を計算する



©Yutaka Matsuo. All rights reserved.

参考文献

[Mitchell 1997] “Machine Learning” McGraw-Hill Science Engineering ISBN:978-0070428072

[Rosenblatt 1957] "The Perceptron—a perceiving and recognizing automaton“ Cornell Aeronautical Laboratory

[Mcculloch & Pitts 1943] "A Logical Calculus of Ideas Immanent in Nervous Activity“ Bulletin of Mathematical Biophysics

[Minsky+ 1969] “Perceptrons: An Introduction to Computational Geometry ” MIT Press ISBN:978-0262631112

[Glorot+ 2011] “Deep Sparse Rectifier Neural Networks”

[Maas+ 2013] “Rectifier Nonlinearities Improve Neural Network Acoustic Models” Computer Science Department, Stanford University, CA 94305 USA

[Hornik 1989] “Multilayer Feedforward Networks are Universal Approximators” eural Networks, Vol. 2, pp. 35Y-366, 198Y

[He+ 2015] “Deep Residual Learning for Image Recognition” arXiv:1512.03385

[Rumelhart+ 1986] “Learning representations by back-propagating errors” Nature

[Cortes+ 1995] “Support-Vector Networks, Machine Learning” , 20(3):273-297, September 1995

[Widrow 1960] “Adaptive Switching Circuits” IRE WESCON Convention Record, 4:96-104

[Amari 1967] “A theory of adaptive pattern classifiers” IEEE Transactions on Electronic Computers:299-307

[Pascanu+ 2012] “On the difficulty of training Recurrent Neural Networks” arXiv:1211.5063

[Hendrycks & Gimpel 2016] “Gaussian Error Linear Units (GELUs)” arXiv:1606.08415

©Yutaka Matsuo. All rights reserved.

©Yutaka Matsuo. All rights reserved.